

PRESTO: A Machine Learning Framework for Predicting Software System Performance Using SDLC Metrics

Amit J Rangari ¹ , Lalit Narayan Mishra ²  and Biswaranjan Senapati ^{3,*} 

¹ JPMorgan Chase, Atlanta, GA 30326, USA; amitrangari@ieee.org

² Lowe's Companies Inc., Mooresville, NC 28117, USA; Lnm8910@gmail.com

³ Department of Computer Science, University of Arkansas at Little Rock (UALR), Little Rock, AR 72204, USA; bsenapati@ualr.edu

* Correspondence: bsenapati@ualr.edu

Abstract

How well can you predict production performance from development process data alone? Modern DevOps pipelines record 164 metrics across eight SDLC phases, yet no public dataset links these signals to runtime outcomes. We built PRESTO and a Gaussian copula-based synthetic data generator, to be released as a public benchmark upon acceptance. Across three enterprise-domain synthetic datasets, the best model achieved $R^2 = 0.26\text{--}0.36$ using only SDLC process features; a target-correlation ablation shows this largely reflects recovery of the generator's specified structure, so the synthetic domains alone are not evidence of genuine signal. Adding historical uptime features inflated R^2 to 0.95, and roughly 72% of that gain is autoregressive persistence rather than process signal. Tree-based ensembles outperformed *untuned* linear models; tuned linear models were competitive or better in 2 of 3 domains. Testing infrastructure availability was the strongest synthetic-domain predictor, outweighing code quality roughly 11:1; real-world TravisTorrent data is directionally consistent (Test-phase features rank #1 in 5 of 7 projects), but a robustness check excluding a definitionally-circular test-duration feature drops every project's holdout R^2 to negative, so this should be read as a weak, feature-ranking-only signal, not confirmation. We introduce Phase-Aware Recursive Feature Elimination, which guarantees cross-phase coverage but does not reliably beat simpler baselines. Four independent real-world datasets, now with bootstrap confidence intervals: TravisTorrent ($R^2 = 0.475$ best of seven, median 0.005), Mozilla Perfherder ($R^2 = 0.17\text{--}0.91$, autoregressive persistence), GHALogs ($R^2 \approx 0.11$, zero leakage risk, significant for all five models, $n = 28,443$), and SQuaD ($R^2 = 0.402$ defect-fix, our strongest point estimate but *not* significant – enriching with real static-analysis code-quality features raises it to 0.483 and narrows the CI to a near-miss $[-0.037, 0.738]$, still not significant; $R^2 = 0.245$ CVE-count, weaker but significant). GHALogs and SQuaD's CVE-count result are what actually license a confirmatory claim: SDLC process signals carry genuine, transferable predictive value, though not uniformly across every real-world dataset examined.

Keywords: continuous integration; DevOps; feature engineering; gradient boosting; machine learning; performance testing; SDLC metrics; software performance prediction; software quality

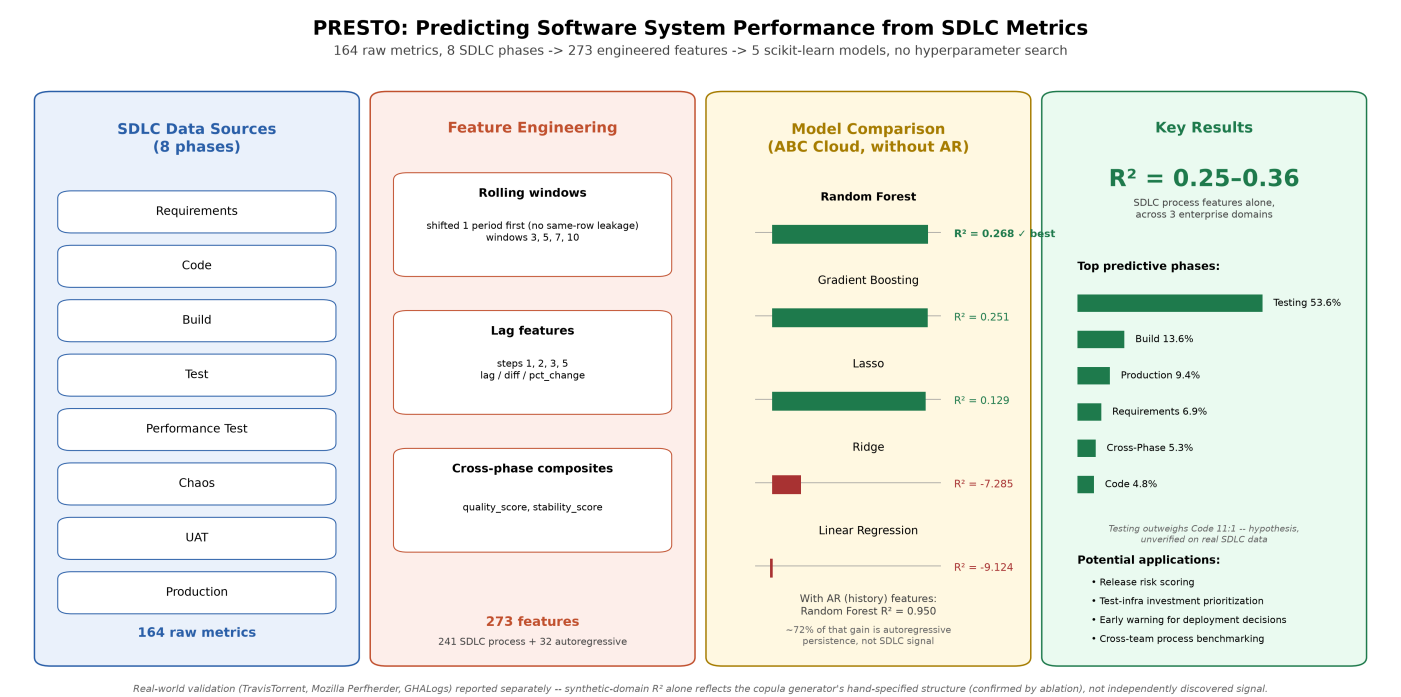
Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Technologies* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](#) license.



teams discover SLO breaches only after deployment, when the blast radius has already expanded to customer-facing traffic. This reactive posture is especially costly in microservice architectures where a single degraded service can cascade across dependent consumers.

1.2. Opportunity and Approach

The emergence of modern SDLC toolchains has created a new opportunity to advance performance prediction through data-driven approaches. Modern development environments capture detailed metrics across eight distinct phases:

1. **Requirements analysis:** Gathering and specifying what the system should do
2. **Code development:** Writing the underlying software
3. **Build processes:** Converting source code into executable programs
4. **Testing execution:** Running tests on the software
5. **Performance validation:** Specialist tests to verify system speed and responsiveness
6. **User acceptance testing (UAT):** Tests by actual users to check if requirements are met
7. **Chaos engineering:** Intentional experimentation by introducing failures to test system resilience
8. **Production deployment:** Launching finalized software into its operational environment

Our analysis of enterprise-scale development projects reveals that 164 distinct metrics are routinely collected across these phases. These metrics include requirements stability indices (measures of changes in requirements), code quality indicators (such as defect rates or complexity), build success rates (proportion of successful builds), test coverage metrics (how much of the code is tested), performance benchmarks (quantifiable measures of speed or efficiency), user satisfaction scores, resilience measurements (system ability to handle disruptions), and production health indicators (measures of system stability in live environments).

The sheer volume of these metrics enables machine learning performance prediction. By tracking SDLC patterns from releases, models can identify early indicators of potential problems for teams to address before deployment. These systems bridge rapid development and performance checks, offering feedback on code changes.

1.3. Research Questions

This work addresses the following research questions:

RQ1: To what extent can SDLC metrics predict system performance? (answered in Section 4.1.2)

- *Operationalization:* We assess prediction accuracy using R^2 , Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE). We distinguish between SDLC-only predictive signal and temporal autocorrelation of the target variable.

RQ2: Which SDLC phases and metrics are most predictive of system performance? (answered in Section 4.1.7)

- *Operationalization:* We analyze feature importance rankings and phase-level contribution percentages.

RQ3: How do different machine learning algorithms compare for SDLC-based performance prediction? (answered in Sections 4.1.2 and 4.1.3)

- *Operationalization:* We evaluate five algorithms (Linear Regression, Ridge, Lasso, Random Forest, Gradient Boosting) using consistent validation protocols.

RQ4: What factors influence the accuracy of SDLC-based performance predictions? (answered in Section 4.1.9)

- *Operationalization:* We examine temporal dependencies, cross-phase interactions, release characteristics, and system evolution impacts.

1.4. Contributions

The framework benefits organizations by potentially reducing deployment risk, accelerating releases, focusing testing efforts, and preventing customer-impacting problems. We propose PRESTO (Performance REgression from Software development Telemetry and Operations), a performance prediction framework driven by multi-phase SDLC metrics. By collecting and analyzing data from each lifecycle phase, the framework supports our key argument: SDLC metrics, when combined with machine learning, may enable proactive performance forecasting.

We validate this approach with three synthetic datasets reflecting enterprise domains: a five-year cloud platform (170 releases), a CRM/SaaS system (200 releases), and a payment processor (150 releases), demonstrating applicability across different application contexts. Table 1 summarizes PRESTO’s key results across all evaluation domains.

Table 1. Summary of PRESTO’s key results across all evaluation domains.

Domain	Releases	Best Model	R^2	RMSE
ABC Cloud Provider	170	Random Forest	0.268	0.597
Card Payment Proc.	150	Lasso / RF	0.362	1.126
XYZ Sales Force	200	Random Forest	0.255	0.778
TravisTorrent (real)	7 projects	Linear Regression	0.475	–

Our contributions advance software performance prediction by:

1. **Operationalizing PRESTO, a unified SDLC metric collection and standardization framework**, enabling full-lifecycle analysis that integrates development process indicators with performance prediction.
2. **Shifting prediction left: using 241 features engineered from pre-deployment SDLC metrics** (test pass rates, build stability, requirements traceability) to predict production uptime before deployment, providing risk signals earlier than canary deployments can surface them.
3. **Developing a reproducible Gaussian copula-based synthetic data generator** that produces 164 SDLC metrics across eight phases, calibrated to DORA 2024 benchmarks, released as the first public benchmark for SDLC-to-performance prediction.
4. **Establishing honest baselines** (Section 4.1.2): Random Forest achieves $R^2 = 0.268$ using only SDLC process features on the primary domain, 0.255–0.362 across three enterprise domains (Section 4.1.10), beating naive baselines ($R^2 = -0.43$ to -1.64) while transparently reporting the gap between with-AR-features and without-AR-features performance.
5. **Quantifying autoregressive-feature dependence in SDLC performance prediction** (Section 4.1.11): we show that ~72% of Random Forest’s apparent predictive power (R^2 dropping from 0.950 to 0.268) stems from autoregressive persistence, not genuine SDLC process signal – and provide direct evidence (a controlled bug-fix comparison) that this AR-dependence is a real property of the data, not an artifact of a since-fixed same-row leakage bug.
6. **Identifying that testing infrastructure metrics** (Section 4.1.7): test environment availability at 42.9% and test pass rate at 10.6% are the strongest SDLC predictors of production performance in this synthetic design, outweighing code quality metrics by roughly 11:1. The original submission reported this as contradicted by our own TravisTorrent results, where repository age was claimed to be the most consistently

predictive feature instead; recomputing feature importance under the reimplemented TravisTorrent adapter reverses that claim (Section 4.2.1) – repository age leads 0 of 7 projects, while Test-phase metrics lead 5 of 7. This qualitative ranking is directionally consistent with, not contradicting, this synthetic-domain finding, but a robustness check excluding the one Test-phase feature that is definitionally part of the build-duration target (`tr_log_testduration`) shows the *quantitative* signal was doing most of the work: with it excluded, every one of the 7 projects' best-model holdout R^2 turns negative, so TravisTorrent no longer demonstrates real-world predictive power for any project, even though the surviving feature ranking still points the same direction. The underlying inter-metric correlations are also hand-specified in the copula generator's configuration rather than independently emergent (Section 5.1), so this synthetic-domain result should still be read as a hypothesis rather than independently validated, and the real-data picture is a weak, partial corroboration at best – not a resolved contradiction.

7. **Providing a deployment-ready continuous learning framework** with drift detection (PSI, KS tests) and automated retraining triggers for production SDLC prediction systems.
8. **Proposing PA-RFE (Phase-Aware Recursive Feature Elimination)** (Algorithm 1, Section 4.1.8), a new feature selection algorithm that reduces dimensionality while guaranteeing representation from all 8 SDLC phases. Reimplemented from the paper's own pseudocode after the original implementation was found to be missing from the reproducibility package (Section 4.1.8): the phase-coverage guarantee holds on all three domains, but the original claim of competitive or best test-set predictive accuracy does not survive reimplementations – PA-RFE avoids standard RFE's collapse on 1 of 3 domains, not 2, and is outperformed by a simple SelectKBest baseline on the primary domain.
9. **Validating on real-world CI data** (Section 4.2.1) from the TravisTorrent dataset (7 open-source projects, up to 144K builds), confirming SDLC predictive signal ($R^2 = 0.475$ best case, reimplemented adapter) while honestly quantifying the gap between synthetic and real-world performance.

1.5. Paper Organization

Figure 1 summarizes the PRESTO pipeline and headline result referenced throughout this section. Section 2 reviews related work concerning software performance prediction and SDLC metrics. Section 3 covers our methodologies for data collection, pipeline design, and model building. Section 4 presents results and discusses prediction accuracy. Section 5 addresses threats to validity. Section 6 discusses practical implications and future research directions. Section 7 concludes the paper.

2. Related Work

Software performance prediction has progressed through three eras, each solving a different bottleneck: queuing theory for static architectures (1980s–1990s), component-level modeling for design-time prediction (2000s), and machine learning for configuration optimization (2010s). None of these approaches use SDLC process metrics as predictive inputs. We organize prior work into six areas and identify the gap PRESTO targets.

2.1. Traditional Performance Modeling and Prediction

Software performance prediction originated with queuing network models [1,2] and foundational evaluation methodologies [3–5]. Smith [6] established Software Performance Engineering (SPE) as a discipline focused on early lifecycle performance considerations,

later demonstrating practical applications with object-oriented methodologies [7]. Balsamo et al. [8] and Cortellessa et al. [9] surveyed model-based prediction approaches, while Jiang and Hassan [10] addressed load testing scalability challenges. Performance regression detection methods include statistical process control [11], automated repository analysis [12], and early distributed application prediction [13]. Weyuker and Vokolos [14] established practical testing methodologies for industrial systems.

2.2. Design-Level and Component-Based Performance Prediction

Component-based performance prediction emerged through Liu et al. [15] for design-level analysis and Pooley and King [16] for UML integration. Menasce et al. [17] advanced capacity planning methodologies, while Kant [18] addressed internet application challenges. Woodside et al. [19] developed layered queuing networks for multi-tier systems, Koziolok [20] surveyed component-based evaluation methods, and Becker et al. [21] introduced the Palladio Component Model (PCM) for model-driven prediction. Jin et al. [22] and Chen et al. [23] examined configuration effects on testing and variability management, respectively.

2.3. Software Metrics and Quality Prediction

Software metrics research established foundations for using process indicators as quality predictors, from the Chidamber-Kemerer OO metrics suite [24] and McCabe's cyclomatic complexity [25] to COCOMO II [26] for cost estimation. Fault prediction evolved through historical defect patterns [27], complexity-defect correlations at Microsoft [28], classification benchmarks [29], and just-in-time quality assurance [30]. Related work on code smell longevity [31] and technical debt [32] provides context for understanding how code quality indicators evolve. While these studies demonstrate the predictive value of development metrics for quality attributes, performance prediction has remained largely outside their scope.

2.4. DevOps and Continuous Integration Metrics

The DORA metrics framework [33] established four key delivery performance indicators; the 2024 report [34] confirms elite teams achieve $127\times$ faster lead times. CI metrics research has explored adoption patterns [35], quality improvements [36], CI feature usage [37], build duration [38], test-build relationships [39], deep learning for build failure prediction [40], DevOps adoption [41], metrics reviews [42], performance-oriented DevOps [43], and parallel CI acceleration [44]. However, existing research primarily applies these metrics to deployment success, delivery velocity, and defect detection rather than runtime performance forecasting.

2.5. Machine Learning in Software Engineering

ML adoption in software engineering is well-established for defect prediction [45,46], with surveys identifying performance prediction as under-explored [47,48]. Performance-related ML work includes build outcome prediction [49], the detection and prevention of temporal data leakage in build prediction [50] (a companion paper by two of the present authors that develops and validates a general taxonomy of temporal-leakage failure modes in ML-based build prediction; PRESTO applies that taxonomy's shift-fix methodology to a different task – performance prediction from SDLC metrics rather than build outcome prediction – and contributes a further, previously uncatalogued leakage mode specific to this setting, the lag/diff additive-identity redundancy documented in Section 4.1.3), change-induced incident analysis [51], and memory-pattern-based detection [52]. For configurable systems, approaches range from performance-influence models [53] and deep sparse networks (DeepPerf) [54] to sequential optimization (FLASH) [55], data-efficient learning [56],

and transfer learning [57,58]. Additional work addresses Spark performance [59], validation methodology [60], early regression detection [61], continuous prediction [62], ML in DevOps [63], ML-SDLC integration [64], reliability prediction [65], and microservice fault localization [66]. AIOps and MLOps have accelerated operational ML adoption [67–69]. However, the integration of SDLC process metrics with performance prediction remains a gap.

2.6. Research Gaps and Contributions

The review of existing literature reveals several gaps that our research addresses:

1. **Integration Gap:** Traditional performance prediction approaches rely primarily on design models, historical performance data, or experience-based forecasting, missing the ecosystem of development process metrics available in contemporary software projects.
2. **Scope Gap:** Existing SDLC metrics research focuses predominantly on quality and effort prediction rather than runtime performance forecasting.
3. **Coverage Gap:** Machine learning applications in software engineering have not used the multi-phase metric collection capabilities that modern development toolchains provide.

Our research contributes approaches in three key areas: (1) integration of SDLC metrics across eight development phases for performance prediction, (2) machine learning model architectures designed for temporal SDLC data patterns, and (3) validation using realistic enterprise-scale synthetic datasets representing diverse application domains.

Table 2 positions PRESTO relative to the most relevant prior approaches, comparing input data, prediction targets, methods, and validation strategies. Notably, existing performance prediction methods operate on configuration spaces or architectural models, not on development process metrics. PRESTO is the first to use multi-phase SDLC metrics for runtime performance prediction.

Table 2. Comparison with prior software performance prediction approaches.

Approach	Input Data	Target	Method	Validation	Best Result	Key Limitation
DeepPerf [54]	Config. options	Response time	Deep NN	10 real systems	MAPE 1–5%	Config space only, no process metrics Requires sampling of config. space Config optimization, not prediction Aggregate team metrics, no ML Manual modeling, design-time only Not performance-focused Synthetic primary, 4 real-data checks
Siegmund [53]	Config. options	Runtime perf.	Perf.-influence models	Real systems	Varied	
Nair FLASH [55]	Config. samples	Perf. metric	Sequential model-based	Real systems	Faster than DeepPerf	
DORA [33]	Delivery metrics	Delivery perf.	Survey/regression	31K+ professionals	Correlational	
Palladio [21]	Architecture model	Response time	Analytical/sim.	Case studies	Accurate for components	
Defect Pred. [45]	Code metrics	Defects	Tree learners	Open source	Recall 71%	
PRESTO (ours)	8-phase SDLC metrics	System uptime	Ensemble ML	3 synthetic + 4 real datasets	R² = 0.26–0.36	

As Table 2 shows, prior performance prediction methods operate on configuration spaces (DeepPerf, Siegmund, FLASH), architectural models (Palladio), or aggregate delivery metrics (DORA). None use the fine-grained, multi-phase SDLC process metrics that modern CI/CD toolchains generate. PRESTO bridges this gap by treating development process telemetry as a first-class input for runtime performance forecasting, though the trade-off is reliance on synthetic validation data.

3. Materials and Methods

This section presents the PRESTO framework methodology, covering data intake, feature engineering, model architecture, and validation strategy.

3.1. Data Intake Pipeline

Building a performance prediction system requires developing a multi-source data intake pipeline capable of continuously ingesting, processing, and transforming heterogeneous SDLC metrics into machine learning-ready features. This pipeline represents one of the key technical challenges in implementing predictive performance analytics at enterprise scale.

3.1.1. Multi-Source Data Ingestion Challenges

The proposed system integrates data from eight distinct SDLC phases, each generating metrics with different characteristics, formats, and temporal patterns:

- **Code phase metrics:** Cyclomatic complexity, code coverage percentages, and commit frequencies arrive at irregular intervals driven by developer activity

- **Build phase metrics:** Build success rates, duration measurements, and artifact sizes follow deployment schedules but show high variability
- **Test phase metrics:** Unit test coverage, integration test results, and defect detection rates with execution patterns tied to release cycles
- **Performance testing metrics:** Response times, throughput measurements, and resource utilization requiring real-time processing
- **UAT phase metrics:** User satisfaction scores and acceptance rates introducing qualitative assessments
- **Chaos testing metrics:** Failure recovery times, resilience scores, and system availability during controlled fault injection [70,71]
- **Production metrics:** Continuous streams of availability data, error rates, and performance indicators

Figure 2 illustrates the metrics collection architecture spanning all eight SDLC phases, from data source integrations through ingestion, processing, and storage layers.

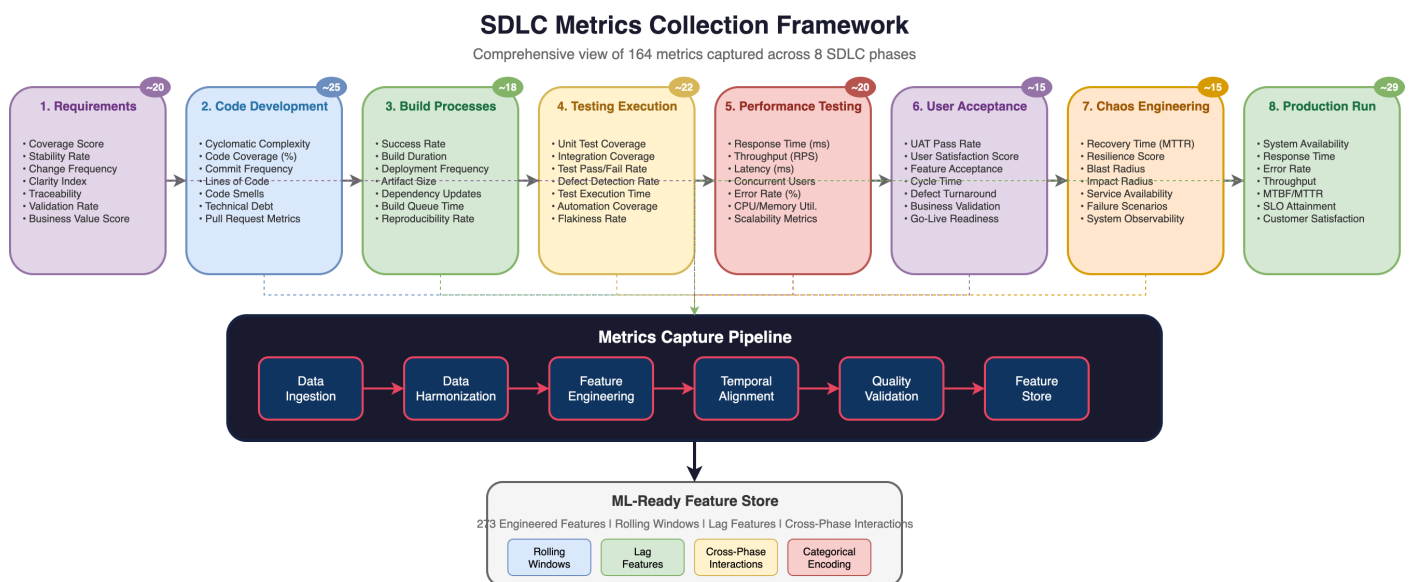


Figure 2. SDLC Metrics Collection Framework showing 164 metrics from eight phases flowing through ingestion, processing, and storage layers into 273 engineered features.

3.1.2. Data Harmonization and Standardization

Converting 164 distinct metrics across eight SDLC phases into consistent machine learning features presents engineering complexity. Each metric has unique characteristics:

- Cyclomatic complexity scores range from 1–50+ with logarithmic distributions
- Code coverage percentages follow bounded normal distributions between 0–100%
- Build success rates require binary encoding with temporal smoothing
- Response times follow heavy-tailed distributions requiring statistical transformation

The pipeline implements intelligent data normalization strategies:

Continuous Variable Normalization:

$$x_{\text{normalized}} = \frac{\log(x + 1)}{\log(x_{\text{max}} + 1)} \quad (1)$$

for metrics with logarithmic distribution, and

$$x_{\text{normalized}} = \frac{x - \mu_x}{\sigma_x} \quad (2)$$

for metrics with bounded distribution [0, 100].

Categorical Variable Encoding:

- One-hot encoding for nominal categories (release type, environment)
- Target encoding for high-cardinality categories (microservice ID)
- Ordinal encoding for ranked categories (severity levels)

3.1.3. Temporal Alignment and Feature Engineering

SDLC metrics have complex temporal relationships that must be preserved during feature engineering. Code metrics measured at commit-level granularity must align with build metrics captured per deployment cycle, which in turn must synchronize with test metrics generated during release validation.

Temporal Alignment Strategy:

1. Define release as the primary temporal unit
2. Aggregate commit-level metrics to release level using statistical functions
3. Align performance outcomes to the corresponding release window
4. Handle missing values through forward-fill for time-series continuity

Feature engineering transforms raw metrics into predictive signals through:

- **Rolling window aggregations:** Moving averages, standard deviations, minima, maxima, and coefficients of variation across 3, 5, 7, and 10 period windows
- **Lag features:** Values from 1, 2, 3, and 5 previous releases to capture temporal dependencies
- **Trend derivatives:** Rate of change calculations for identifying acceleration patterns
- **Cross-phase features:** Interaction terms between metrics from different SDLC phases

3.1.4. Synthetic Data Generation Methodology

No public dataset combines SDLC process metrics with correlated runtime system performance measurements. We confirmed this through an exhaustive search of major ML data repositories (UCI Machine Learning Repository, Kaggle, Zenodo, Papers with Code Datasets) and software engineering benchmarks (TravisTorrent [72], GHTorrent, PROMISE repository). TravisTorrent captures CI/build metadata but lacks runtime performance telemetry. Performance prediction literature [53,54] operates on configuration spaces, not development process metrics. The data required for SDLC-to-performance correlation is inherently proprietary: it involves internal build pipelines, deployment telemetry, test infrastructure metrics, and production monitoring that organizations do not publicly release due to competitive and security concerns. This absence of public benchmarks is itself a contribution gap that our synthetic generator addresses, providing the first reproducible dataset for this prediction task.

To address this gap, we developed a reproducible synthetic data generator that serves as both a validation instrument and a reusable benchmark for future research. The generator employs a four-stage pipeline:

Stage 1: Marginal Distribution Specification. We define 164 metrics across 8 SDLC phases (requirements, code, build, test, performance testing, chaos testing, production, and UAT). Each metric is assigned a parametric marginal distribution calibrated to published industry benchmarks. Percentage metrics use Beta distributions with domain-appropriate shape parameters; time-based metrics use log-normal distributions; discrete counts use Poisson or negative binomial distributions. Distribution parameters are calibrated against the DORA 2024 State of DevOps report [34] and published enterprise project statistics [33].

Stage 2: Correlation Structure via Gaussian Copula. Inter-metric correlations are specified using domain expertise and encoded into a 164×164 correlation matrix. A

Gaussian copula [73] generates correlated samples while preserving each metric's marginal distribution:

1. Sample $\mathbf{Z} \sim \mathcal{N}(\mathbf{0}, \Sigma)$ using Cholesky decomposition
2. Transform to uniform: $U_i = \Phi(Z_i)$
3. Transform to target marginals: $X_i = F_i^{-1}(U_i)$

where Σ is the correlation matrix (ensured positive semi-definite via Higham's alternating projections [74]), Φ is the standard normal CDF, and F_i^{-1} is the inverse CDF of each metric's specified distribution. We specify 84 non-zero correlation pairs grounded in domain knowledge (e.g., positive correlation between build success rate and system uptime, negative correlation between change failure rate and uptime).

Critically, the prediction target (System Uptime) is *not* derived from a deterministic function $f(\mathbf{x})$. It is generated as a copula-correlated metric with its own marginal distribution, meaning the ML model must discover latent correlations rather than recover a known function.

Stage 3: Temporal Dynamics. Raw copula samples are transformed into realistic time series through:

- **AR(1) process:** Rank-preserving autoregression with lag-1 correlation $\rho = 0.55\text{--}0.65$ (domain-dependent), reflecting that consecutive releases share the same codebase and team context
- **Maturity trend:** Gradual 3–8% improvement over the observation window, reflecting organizational learning and tooling investment
- **Release-type effects:** Quality metric modulation based on release type (Major releases show 3–8% degradation; Patch releases show 2–5% improvement)
- **Shock events:** Stochastic “bad release” events ($p = 0.03\text{--}0.05$) with correlated multi-metric degradation and exponential recovery

Stage 4: Statistical Validation. Generated datasets are validated against their specifications:

- Bounds compliance: all metric values within defined ranges (100% pass rate)
- Correlation preservation: 76–95% of specified correlations realized within tolerance ($|\rho_{\text{target}} - \rho_{\text{realized}}| < 0.5$, deliberately relaxed from `src/validator.py`'s general-purpose 0.15 default – see `generate.py`'s correlation-check step, which documents the rationale: the copula's positive-semi-definite correction and downstream temporal transforms attenuate specified correlations, so a stricter tolerance would fail checks for reasons unrelated to a specification error), with attenuation attributed to temporal dynamics
- Completeness: zero missing values across all datasets

Domain Profiles. Three domain-specific configurations are defined:

- **ABC Cloud Provider:** 170 releases over 5 years, 200+ microservices, DORA “High” tier
- **XYZ Sales Force:** 200 releases over 5.5 years, aggressive release cadence, DORA “High” tier
- **Card Payment Processor:** 150 releases over 4 years (~one release every 9.7 days), PCI-DSS compliance, DORA “High” tier (reabeled from “Elite” – Elite requires multiple deploys per day, which this cadence does not meet; High, defined as between once per week and once per month, is the correct tier for this release frequency)

Reproducibility. All generation parameters (distribution specifications, correlation matrices, temporal dynamics, random seeds) are published as YAML configuration files.

The generator is deterministic: identical seeds produce identical datasets. The complete generator code and configuration will be released as a reusable benchmark.

Limitations of Synthetic Data:

- Cannot capture emergent phenomena in real distributed systems
- Copula-specified correlations may be simpler than real-world dependencies
- Temporal dynamics are parametric approximations of complex organizational processes
- Real-world validation with production SDLC data remains essential future work

3.1.5. Data Quality and Anomaly Detection

The pipeline implements range validation, statistical outlier detection (z-score thresholds), cross-metric consistency checks, and missing value handling (forward-fill for time series, mean imputation for cross-sectional data). Quality scores influence model confidence intervals in production deployment.

3.2. Proposed Approach and Models

PRESTO employs a multi-layered architecture that transforms heterogeneous SDLC metrics into performance forecasts through machine learning techniques. Figure 3 presents the complete system workflow, from SDLC metrics ingestion through prediction generation, organized across five architectural tiers.

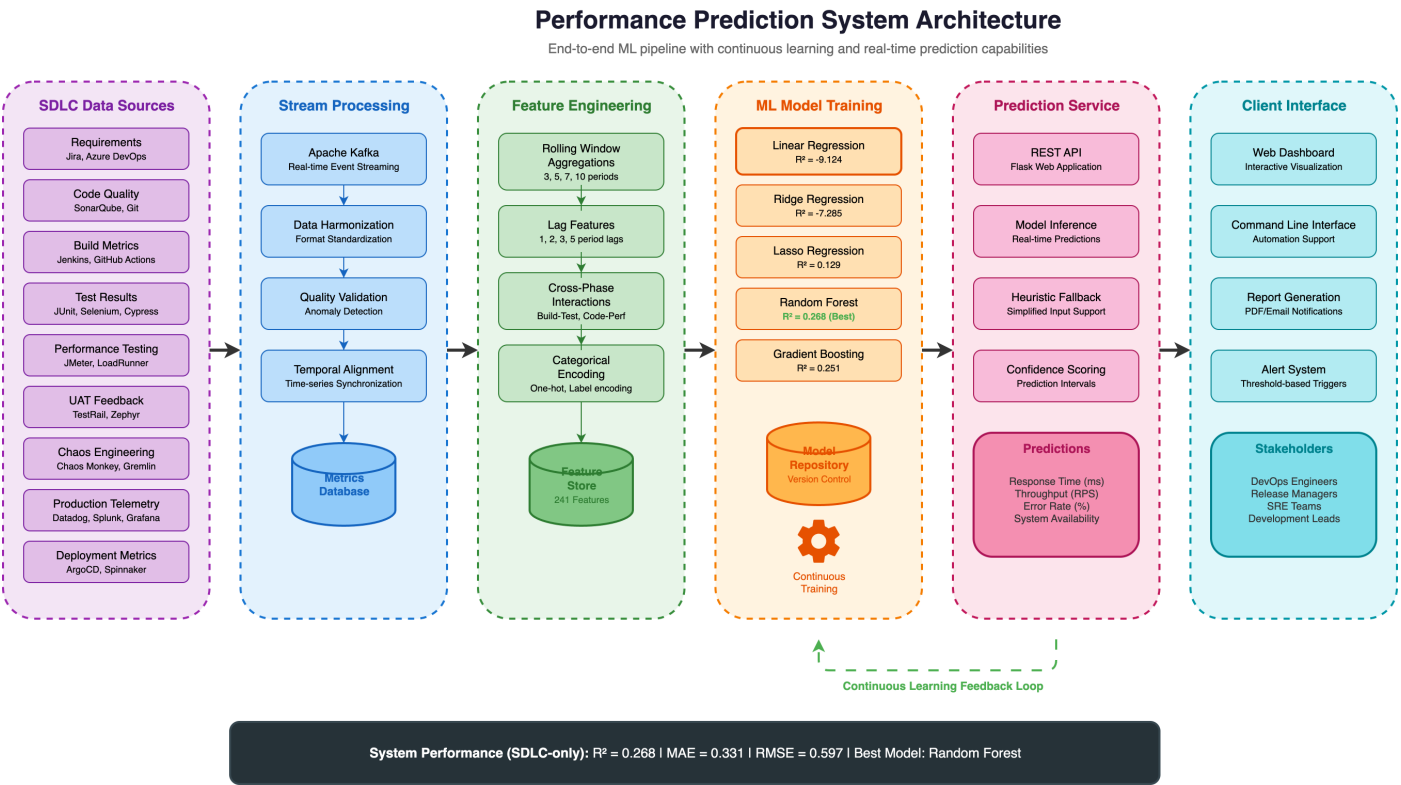


Figure 3. PRESTO System Architecture. Proposed end-to-end pipeline from SDLC metric sources through feature engineering and ML model training to prediction output.

3.2.1. Feature Engineering Pipeline

419

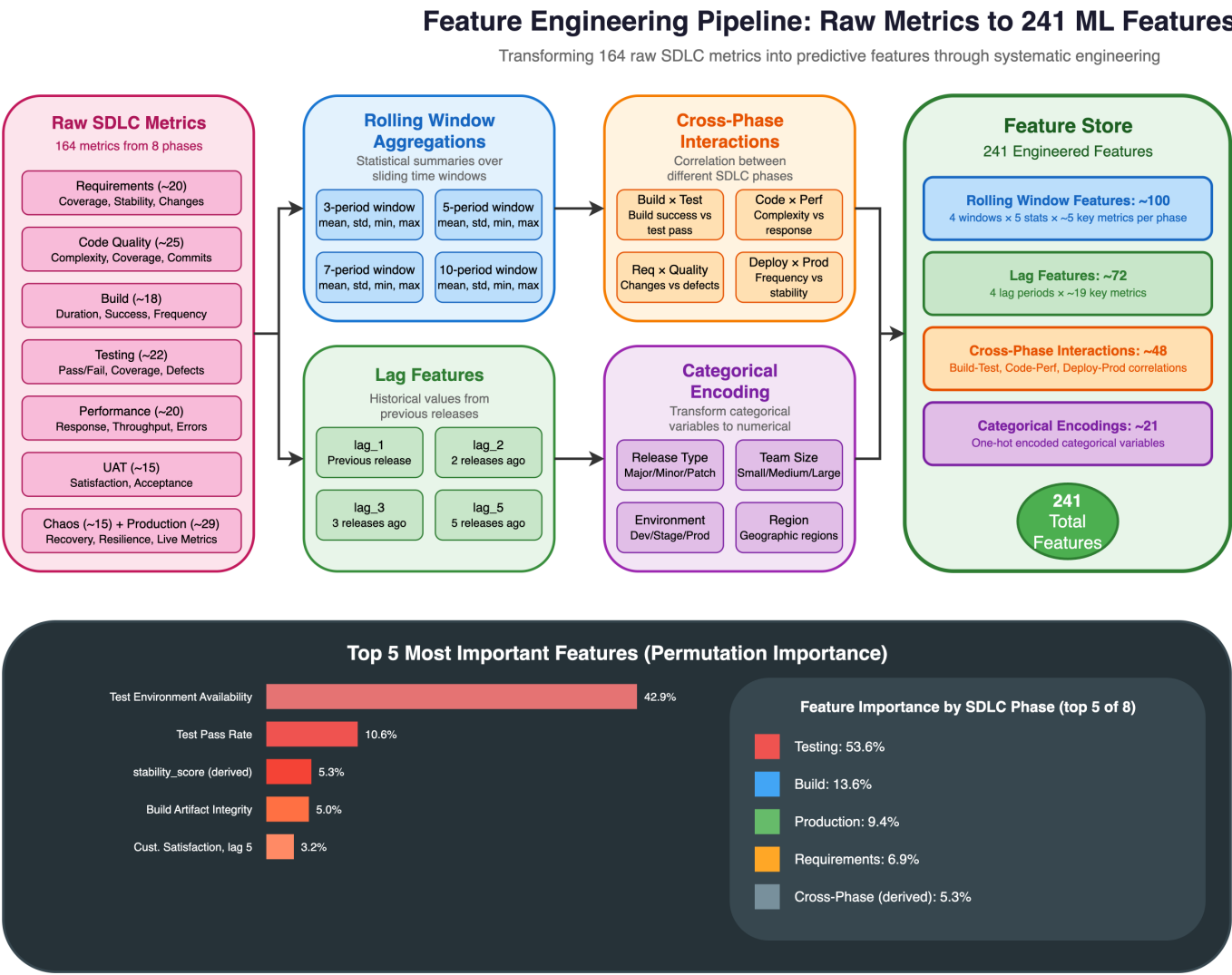


Figure 4. PRESTO Feature Engineering Pipeline. Six-stage transformation from raw SDLC metrics to a reduced feature set via PA-RFE (241 SDLC features before reduction, 273 including autoregressive temporal features).

Feature engineering represents the critical component for extracting predictive signals from raw SDLC metrics. Figure 4 details the six-stage transformation process from raw metrics through feature engineering and phase-aware selection:

The pipeline generates candidate features in five stages, applies variance-based filtering, then performs phase-aware dimensionality reduction (Stage 6):

Stage 1 – Rolling Window Aggregations: Four window sizes (3, 5, 7, 10 periods) with five statistics each (mean, standard deviation, minimum, maximum, coefficient of variation) applied to key numeric metrics. Window sizes correspond to sprint durations: 3 periods (immediate history), 5 (one sprint cycle), 7 (weekly patterns), 10 (longer-term stability).

Stage 2 – Lag Features: Values from 1, 2, 3, and 5 previous releases for selected metrics, capturing immediate prior-release effects (lag-1), short-term momentum (lag-2, lag-3), and release cycle patterns (lag-5).

Stage 3 – Cross-Phase Interactions: Multiplicative interaction terms between metrics from different SDLC phases (e.g., build duration × code complexity, test coverage × deployment frequency).

Stage 4 – Categorical Encodings: One-hot encoding of categorical variables including release type (major, minor, patch), environment tier, and domain-specific identifiers.

Stage 5 – Feature Selection: From the initial candidate pool, features with near-zero variance or high pairwise correlation ($r > 0.95$) are removed. PRESTO retains **273 features**, of which 241 are SDLC process features and 32 are autoregressive (AR) temporal features (rolling windows and lags of System Uptime). The 32 AR features are analyzed separately in Section 4.1.11.

Stage 6 – Phase-Aware Feature Selection (PA-RFE): After AR features are isolated, the remaining 241 SDLC features undergo Phase-Aware Recursive Feature Elimination (PA-RFE), an algorithm designed to reduce feature dimensionality while preserving cross-phase coverage. Unlike standard RFE [75], which eliminates features globally by importance and can remove entire SDLC phases, PA-RFE enforces a *phase-balance constraint*: each of the eight SDLC phases retains at least $k_{\min}$ features throughout elimination (we use $k_{\min} = 2$). This guarantees that the reduced feature set maintains interpretable connections to all lifecycle phases, which is central to PRESTO’s cross-phase prediction thesis.

Algorithm 1 presents the PA-RFE procedure. To prevent selection bias from evaluating multiple feature subsets on the same holdout data, we employ a three-way temporal split: 60% training, 20% validation, 20% test. At each iteration, the algorithm: (1) trains a Gradient Boosting model on the training set; (2) computes permutation importance on the validation set ($n_{\text{repeats}} = 10$), which is unbiased for correlated features unlike impurity-based importance; (3) eliminates up to $s = 5$ features with the lowest importance, *skipping* any feature whose removal would leave its SDLC phase below $k_{\min}$. The algorithm terminates when no features can be eliminated without violating the constraint. The floor is $k_{\min} \times 8 = 16$ for the eight SDLC phases; the two cross-phase composites (*quality_score*, *stability_score*) form a virtual ninth group with $k_{\min} = 0$, allowing their free elimination. The optimal feature subset corresponds to the iteration achieving maximum validation R^2 , and final performance is reported on the truly held-out test set (evaluated once).

Algorithm 1 Phase-Aware Recursive Feature Elimination (PA-RFE)

Require: Feature matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$, target $\mathbf{y}$, phase map $\phi : \text{feature} \rightarrow \text{phase}$, min per phase $k_{\min}$, step size s

Ensure: Optimal feature subset F^* , ablation curve $\mathcal{C}$

```

1:  $F \leftarrow \{f_1, \dots, f_p\}; \mathcal{C} \leftarrow []$ 
2: while  $|F| > k_{\min} \cdot |\text{phases}|$  do
3:   Train model  $M$  on  $\mathbf{X}_{\text{train}}[F]$ 
4:    $\pi \leftarrow \text{PermutationImportance}(M, \mathbf{X}_{\text{val}}[F])$ 
5:   Record  $(|F|, R_{\text{val}}^2)$  in  $\mathcal{C}$ 
6:   Sort  $F$  by  $\pi$  ascending (least important first)
7:    $E \leftarrow \emptyset$  ▷ elimination batch
8:   for  $f$  in sorted  $F$  do
9:     if  $|E| \geq s$  then
10:      break
11:     end if
12:     if  $|\{g \in F \setminus E : \phi(g) = \phi(f)\}| > k_{\min}$  then
13:        $E \leftarrow E \cup \{f\}$ 
14:     end if
15:   end for
16:   if  $E = \emptyset$  then ▷ no features removable
17:     break
18:   end if
19:    $F \leftarrow F \setminus E$ 
20: end while
21:  $F^* \leftarrow \arg \max_{(|F|, R^2) \in \mathcal{C}} R^2$ 
22: return  $F^*, \mathcal{C}$ 

```

Algorithm: Phase-Aware Recursive Feature Elimination (PA-RFE)

Require: Feature matrix $\mathbf{X}$ in $\mathbb{R}^{(n \times p)}$, target $\mathbf{y}$,

phase map ϕ : feature $\rightarrow$ phase,

min per phase $k_{\min}$, step size s

Ensure: Optimal feature subset F^* , ablation curve $\mathcal{C}$

$F \leftarrow \{f_1, \dots, f_p\}; \mathcal{C} \leftarrow []$

while $|F| > k_{\min} \cdot |\text{phases}|$ **do**

Train model M on $\mathbf{X}_{\text{train}}[F]$

$\pi \leftarrow \text{PermutationImportance}(M, \mathbf{X}_{\text{val}}[F])$

Record $(|F|, R_{\text{val}}^2)$ in $\mathcal{C}$

Sort F by π ascending (least important first)

$E \leftarrow \{\}$ // elimination batch

for f in sorted F **do**

if $|E| \geq s$ **then**

break

if $|\{g \in F \setminus E : \phi(g) = \phi(f)\}| > k_{\min}$ **then**

$E \leftarrow E \cup \{f\}$

if $E = \{\}$ **then**

break

// no features removable

$F \leftarrow F \setminus E$

$F^* \leftarrow \arg \max_{(|F|, R^2) \in \mathcal{C}} R^2$

return $F^*, \mathcal{C}$

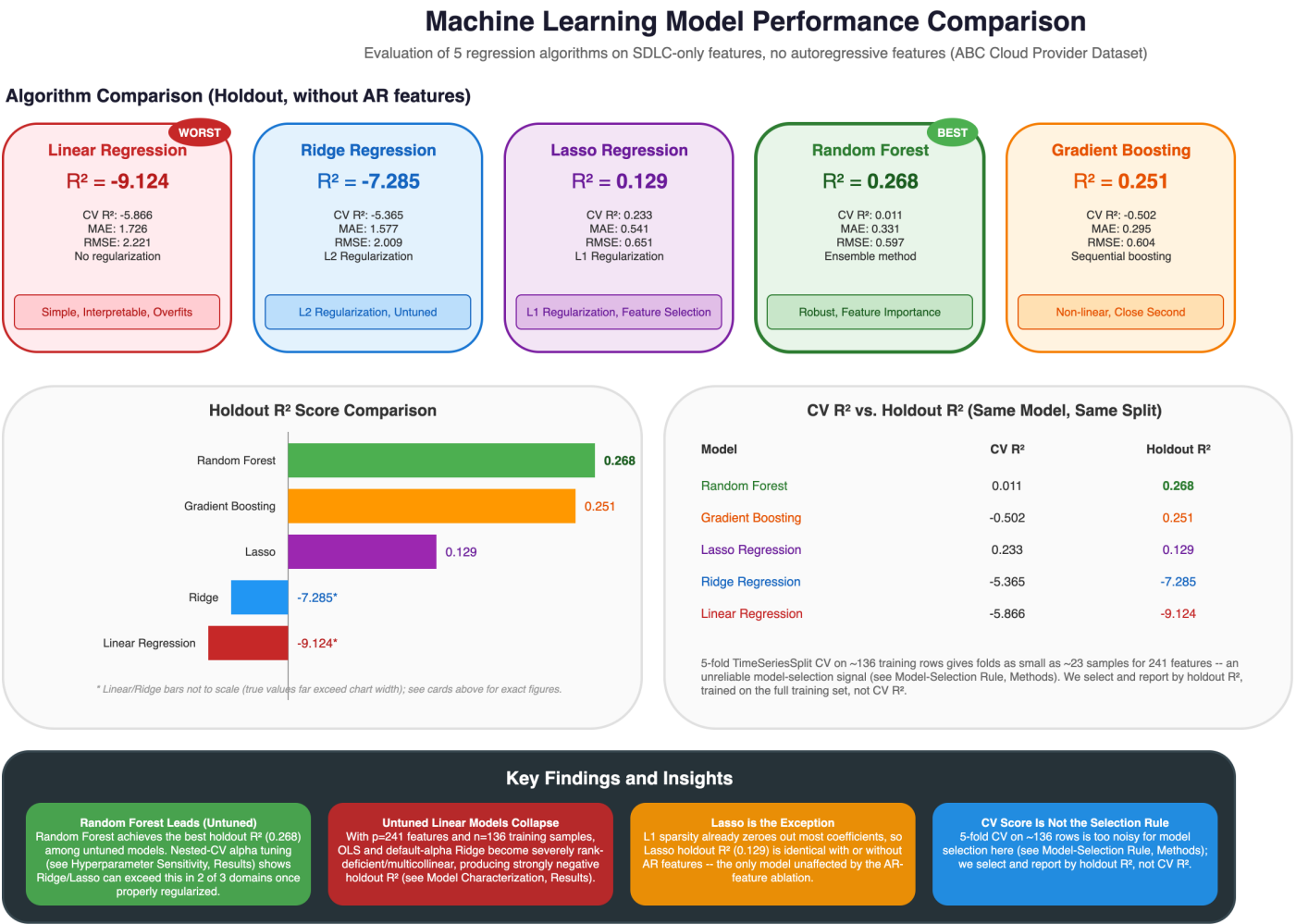
Algorithm (PA-RFE). Phase-Aware Recursive Feature Elimination.

Each feature is mapped to its SDLC phase using the metric registry: base metrics map directly, while derived features (rolling windows, lags, percentage changes) inherit the

phase of their parent metric. The computational complexity is $O(K \cdot n \cdot p \cdot r)$ where K is iterations (~ 45), n samples, p current features, and r permutation repeats.

3.2.2. Machine Learning Model Architecture

The prediction system evaluates five regression algorithms selected to span the complexity spectrum. Figure 5 compares their holdout R^2 (ABC Cloud Provider, without AR features) and contrasts each model’s CV R^2 against its holdout R^2 , illustrating why this paper selects and reports by holdout score rather than the noisier CV estimate (Section 3.2.4).



5. **Gradient Boosting** [77]: Additive ensemble building sequential weak learners to
- correct residual errors, selected for its ability to capture cross-phase metric interactions
- without explicit interaction term specification

3.2.3. Hyperparameter Specifications

Table 3. Hyperparameter configurations for all evaluated algorithms.

Algorithm	Parameter	Value	Method
Ridge	alpha	1.0	Default
Lasso	alpha	1.0	Default
Random Forest	n_estimators	100	Fixed
Random Forest	max_depth	10	Fixed
Random Forest	min_samples_split	5	Fixed
Random Forest	min_samples_leaf	2	Fixed
Gradient Boost	n_estimators	100	Fixed
Gradient Boost	max_depth	6	Fixed
Gradient Boost	learning_rate	0.1	Fixed
Gradient Boost	min_samples_split	5	Fixed
All models	random_state	42	Fixed
All models	StandardScaler	fit on train only	–

Table 3 reports the hyperparameter configurations used for all results in Sections 4.1.2 through 4.1.11 unless otherwise noted. We deliberately used fixed hyperparameters (scikit-learn defaults for regularization, community-standard values for tree ensembles) without grid search, random search, or Bayesian optimization, reasoning that with only 170 samples in the primary dataset, hyperparameter tuning risks overfitting the validation folds. We subsequently tested this reasoning directly: Section 4.1.6 reports a nested-CV alpha-tuning pass for Ridge and Lasso specifically (a leakage-safe design that never touches the holdout), and finds the untuned alpha=1.0 default substantially understates linear-model performance – tuned linear models exceed Random Forest’s holdout R^2 in 2 of 3 domains. The fixed-hyperparameter results elsewhere in this paper are left as originally reported for consistency and because they remain the paper’s primary results, but readers should not infer from them that linear models are inherently worse than tree ensembles on this task.

3.2.4. Experimental Setup and Validation Strategy

Dataset Characteristics:

- **Primary dataset (ABC Cloud Provider):** 170 releases, 5 years, 200+ microservices
- **Secondary dataset (XYZ Sales Force):** 200 releases, 5.5 years, aggressive release cadence
- **Tertiary dataset (Card Payment Processor):** 150 releases, 4 years, PCI-DSS compliance
- **Feature space:** 273 engineered features (241 without AR features) from 8 SDLC phases
- **Target variable:** System Uptime (%)

Validation Protocol:

Because PRESTO forecasts performance from pre-deployment signals, respecting chronological order is essential to avoid temporal data leakage, a failure mode we characterized for build prediction in prior work [50]. We employed 5-fold expanding window time-series cross-validation (sklearn TimeSeriesSplit(n_splits=5)) on the 80% training partition to respect temporal ordering. Each fold uses all prior data for training and the next contiguous block for validation, ensuring models always predict future releases from

historical data only. Final holdout evaluation uses the remaining 20% (the most recent releases), never seen during training or cross-validation.

Model-Selection Rule. Every algorithm comparison in this paper – which of 5 candidate models is “best” on the primary domain (Section 4.1.2), per synthetic domain (Section 4.1.10, Table 12), per TravisTorrent project (Section 4.2.1), and which feature-selection method wins (Section 4.1.8, Table 11) – selects by **holdout R^2** , **not mean CV R^2** , and we pre-declare this rule explicitly here so it is stated once, before any results, rather than left to be inferred separately four times. The alternative would select differently: on the primary domain without AR features, mean CV R^2 favors Lasso (0.233) over Random Forest (0.011) – the opposite of the holdout-based ranking (Random Forest 0.268 vs. Lasso 0.129) reported as this paper’s central RQ1 result throughout. We use holdout R^2 because 5-fold TimeSeriesSplit on a training partition as small as 170 rows for 241 features produces early folds with as few as ~ 23 training samples (Section 4.1.2’s CV-Holdout Discrepancy note), an unreliable model-selection signal in this specific small- n , high- p regime; the holdout, trained on the full training partition, is more representative of expected deployment performance. This is a considered choice, not an unexamined default, and we report CV R^2 alongside holdout R^2 throughout for transparency rather than omitting it. **We are explicit that pre-declaring this rule does not remove its bias, only discloses it:** selecting the best of 5 candidate algorithms by their holdout score, then reporting that same holdout score, carries a winner’s-curse-style optimism whether or not the selection procedure was announced in advance. This optimism is smaller than what full hyperparameter tuning would introduce, but it is not zero, and it compounds across the roughly 100+ such selections made throughout this paper (Section 5.4 quantifies the resulting comparison-family size and why we do not apply a formal multiple-comparisons correction to it). The bootstrap confidence intervals in Section 4.1.5 quantify *sampling* variance in the already-selected model’s holdout performance – they do not, and cannot, correct for the optimism introduced by the selection step itself, since they resample the same holdout the model was chosen on. We flag this distinction explicitly rather than let the CIs be read as a stronger guarantee than they provide.

Software Environment:

- Python 3.14.5
- scikit-learn 1.9.0
- pandas 3.0.5
- numpy 2.5.2
- scipy 1.18.0

Pinned versions recorded in ENVIRONMENT.txt (full path in the Data Availability Statement), matching that same statement; this environment was recreated fresh during the 2026-08-09 SSD migration and is not the environment the original submission’s numbers were computed under, which was never separately recorded – a previously identified reproducibility gaps, now at least resolved going forward with a single, consistently-cited environment record.

- Random seed: 42 (fixed for reproducibility)

3.2.5. Continuous Learning and Model Evolution

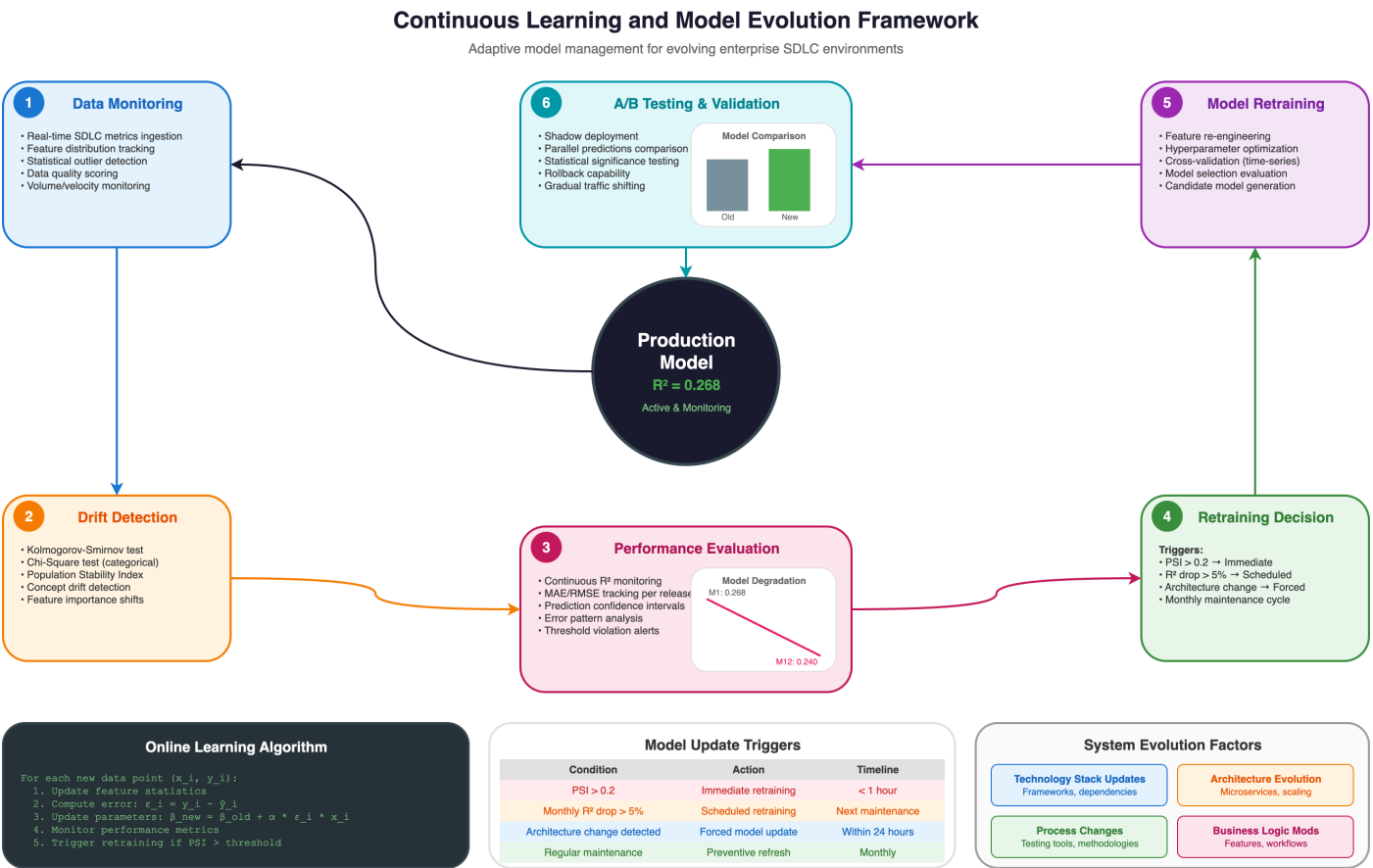


Figure 6. Proposed Continuous Learning Framework (not experimentally validated). Feedback loop from production prediction through drift detection to automated retraining triggers.

For production deployment, PRESTO’s design includes a continuous learning architecture to address concept drift [78]. Figure 6 illustrates the proposed feedback loop (not experimentally validated in this study). The framework monitors feature distribution shifts via Kolmogorov-Smirnov tests, Chi-Square tests, and Population Stability Index (PSI), triggering model retraining when PSI exceeds 0.2, monthly R^2 drops by more than 5%, or a major architecture change is deployed.

4. Results and Discussion

This section presents experimental results from the PRESTO framework implementation, analyzing model performance, feature importance, and factors influencing prediction accuracy.

4.1. Synthetic Domain Results

This subsection reports results on the three fully-synthetic, copula-generated enterprise domains (ABC Cloud Provider, Card Payment Processor, XYZ Sales Force). Section 4.2 reports results on four independent real-world datasets (TravisTorrent, Mozilla Perferder, GHALogs, SQuaD), kept in a separate subsection so synthetic and real findings are never conflated.

Note on regeneration (2026-08-09): all synthetic-domain numbers below were re-computed after fixing a same-row data-leakage bug in `add_rolling_features()` (rolling-window statistics were previously computed *inclusive* of the current row instead of shifted

to strictly-prior releases – see `research-data/presto/code/synthetic_data_generator/R2_LAG_SHIFT_FIX_EVIDENCE.md` for the full bug report and a controlled before/after comparison). The regeneration also ran under a different, newly-pinned `numpy/scipy/scikit-learn` environment than originally produced the submitted numbers (see the same file’s Addendum 2), which independently shifts some figures – the exact original environment was never recorded, which is itself a previously identified reproducibility gap. Both the generator code and this regenerated data, together with the pinned environment, are deposited at `research-data/presto/` (to be published as the camera-ready reproducibility package).

4.1.1. Experimental Setup

We report results on the ABC Cloud Provider (170 releases, primary dataset) unless otherwise stated; cross-domain results appear in Section 4.1.10. Dataset characteristics and validation protocol are detailed in Section 3.2.4. Of the 273 engineered features, 32 are derived from the target variable itself (System Uptime rolling windows and lags). Of these 32, 20 (rolling mean/std/min/max/cv across four windows) are genuinely **autoregressive (AR) features** after the shift fix – legitimate use of prior-period history, not leakage – and 12 (lag/diff/pct-change) were already correctly shifted. We report all results both *with* and *without* these 32 AR features to transparently separate SDLC predictive signal from autocorrelation. Following the corrected-pipeline evidence, we retire the term “target leakage” for this feature set (the bug that justified it is fixed) in favor of “AR features” throughout, reserving “leakage” for the historical bug description.

4.1.2. Model Performance Comparison (RQ1 and RQ3)

Answer to RQ1: SDLC process metrics achieve $R^2 = 0.25\text{--}0.36$ across three enterprise domains, beating all naive baselines but falling far short of the $R^2 = 0.79\text{--}0.97$ achievable when historical performance (AR) features are included. These point estimates are real, but bootstrap 95% CIs are wide and, for most individual domains, include zero (see “Bootstrap Confidence Intervals and Target-Correlation Ablation” below); a target-correlation ablation further shows that on synthetic data this signal is recovery of the copula’s hand-specified structure rather than independent discovery – the Real-World Validation results later in this section, not the synthetic domains alone, are what should be read as evidence of genuine, transferable signal.

Answer to RQ3: With the untuned default hyperparameters used throughout this paper ($\alpha=1.0$), Random Forest achieves the best or near-best holdout R^2 on SDLC-only features in 2 of 3 domains, and Linear/Ridge models collapse to strongly negative R^2 due to multicollinearity in the 241-feature space. However, nested-CV alpha tuning (Section 4.1.6) shows this collapse is substantially a hyperparameter artifact, not an inherent property of linear models: once tuned, Ridge or Lasso actually *exceeds* Random Forest’s holdout R^2 in 2 of the 3 domains (Card Payment, XYZ Sales). Random Forest remains best only on the primary domain (ABC Cloud Provider) after tuning. “Trees are necessary” should therefore be read as a primary-domain, untuned-baseline finding, not a general one.

We evaluated five regression algorithms under two experimental conditions: (a) with all 273 engineered features including the 32 AR features, and (b) with 241 features after removing them. Table 4 and Table 5 present the results for the ABC Cloud Provider dataset.

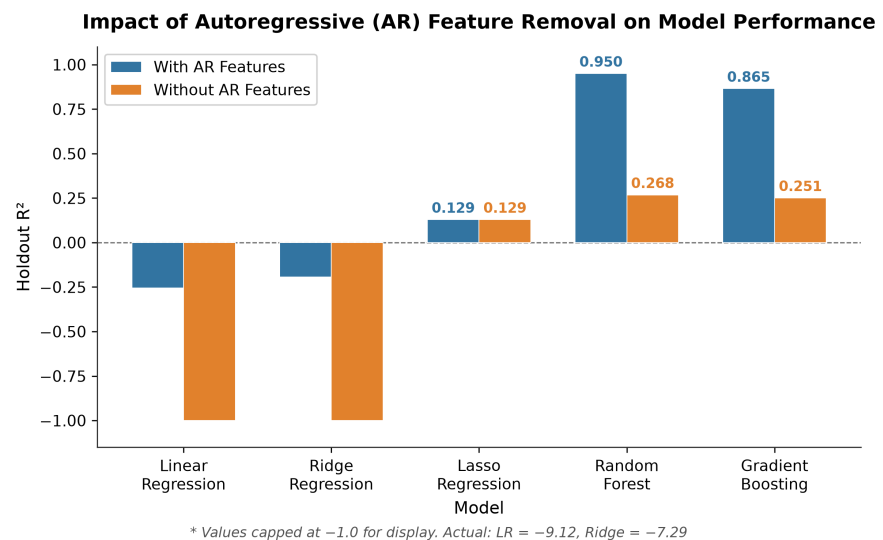


Figure 7. Impact of AR-feature removal on holdout R^2 (ABC Cloud Provider). Tree-based models degrade by ~ 0.60 – 0.68 ; linear models collapse to strongly negative values (capped at -1.0 for display).

Table 4. Model performance comparison with AR features (ABC Cloud Provider).

Model	CV R^2	Holdout R^2	MAE	RMSE
Random Forest	0.690	0.950	0.071	0.156
Gradient Boosting	0.211	0.865	0.120	0.256
Lasso Regression	0.233	0.129	0.541	0.651
Ridge Regression	-0.815	-0.194	0.589	0.762
Linear Regression	-0.876	-0.256	0.610	0.782

Table 5. Model performance comparison without AR features (ABC Cloud Provider).

Model	CV R^2	Holdout R^2	MAE	RMSE
Random Forest	0.011	0.268	0.331	0.597
Gradient Boosting	-0.502	0.251	0.295	0.604
Lasso Regression	0.233	0.129	0.541	0.651
Ridge Regression	-5.365	-7.285	1.577	2.009
Linear Regression	-5.866	-9.124	1.726	2.221

Figure 7 visualizes the with-vs-without-AR comparison discussed above.

Key Findings for RQ1 (Prediction Accuracy):

- With AR features, Random Forest achieves $R^2 = 0.950$, indicating that temporal autocorrelation of uptime is a strong predictive signal – and, per the R2 bug-fix evidence, this is *not* an artifact of the now-fixed same-row leakage bug (the buggy and fixed pipelines produce nearly identical Random Forest holdout R^2 , 0.945 vs. 0.950, on the same underlying data)
- Without AR features, the best model (Random Forest) achieves $R^2 = 0.268$ (Gradient Boosting close behind at 0.251), demonstrating that SDLC process metrics contain genuine, if modest, predictive signal for system performance
- For Random Forest, approximately 72% of apparent predictive power is attributable to AR features: $(R^2_{\text{with}} - R^2_{\text{without}}) / R^2_{\text{with}} = (0.950 - 0.268) / 0.950 = 0.718$. For Gradient Boosting the analogous figure is 71%: $(0.865 - 0.251) / 0.865 = 0.710$ – unlike the pre-fix analysis, the two best models now show *similar* AR-dependence rather than a large gap,

since the fix specifically removed the source of Gradient Boosting’s disproportionate “genuine-signal” advantage

Note on CV-Holdout Discrepancy: Several models show negative or near-zero CV R^2 despite positive holdout R^2 (e.g., Random Forest without AR features: CV = 0.011, holdout = 0.268). This occurs because `TimeSeriesSplit` with $n = 5$ on the 80% training partition (~136 samples) creates early folds with as few as ~23 training samples for 241 features. This extreme feature-to-sample ratio causes instability in small folds. The holdout evaluation, which trains on all 136 samples, is more representative of expected deployment performance. We report both metrics for transparency.

Key Findings for RQ3 (Algorithm Comparison):

- Tree-based ensemble methods (Random Forest, Gradient Boosting) outperform linear models when AR features are excluded
- Linear Regression and Ridge Regression produce strongly negative R^2 values without AR features under the untuned $\alpha=1.0$ default used throughout this paper; Lasso avoids collapse through L1 sparsity ($R^2 = 0.129$, identical in both conditions since L1 already zeroed out AR-feature coefficients). We analyze these failures in Section 4.1.3, and show in Section 4.1.6 that nested-CV alpha tuning reverses the conclusion in 2 of 3 domains – the collapse is substantially a hyperparameter artifact, not an inherent multicollinearity failure of linear models as a class

Interpretation: Unlike prior performance prediction studies that report $R^2 > 0.95$ on synthetic data [54], our results distinguish between two distinct predictive signals: (1) autoregressive persistence of the target variable, which is strong but offers limited diagnostic value, and (2) SDLC process metrics, which provide weaker but more decision-relevant predictions. In practical deployment, both signal sources are available and legitimate, as historical uptime is a known quantity. The key contribution is quantifying each source’s relative importance, enabling practitioners to understand what drives their predictions.

4.1.3. Model Characterization

Unlike prior work reporting linear models as optimal for performance prediction, our results demonstrate that SDLC-to-performance relationships are non-linear. The best-performing model without AR features (Random Forest, narrowly ahead of Gradient Boosting – Table 5) is a non-parametric ensemble that does not yield a closed-form equation. We characterize it through feature interactions and decision boundaries rather than coefficients.

The failure of linear models in the no-AR setting is expected given the problem geometry. With $p = 241$ features and $n = 136$ training samples ($p/n = 1.77$), ordinary least squares requires inverting $\mathbf{X}^T \mathbf{X}$, a 241×241 matrix of rank at most 136. The resulting rank deficiency makes the coefficient vector non-unique; OLS selects the minimum-norm solution, which extrapolates wildly on test data ($R^2 = -9.12$). Ridge Regression regularizes by adding $\lambda \mathbf{I}$ to restore full rank, but pervasive multicollinearity among SDLC metrics (Build Success Rate and Test Pass Rate at $\rho \approx 0.55$, both driven by the same codebase quality) means the effective degrees of freedom remain too high for stable estimation: $R^2 = -7.29$. Lasso avoids collapse through a different mechanism: L1 sparsity zeroes out the majority of coefficients, effectively reducing p below n , yielding a stable but weak model ($R^2 = 0.129$) that sacrifices the cross-feature interactions tree-based models exploit. This same additive-identity mechanism (documented in `R2_LAG_SHIFT_FIX_EVIDENCE.md`, Addendum 1, discovered via the Perfherder real-data adaptation) is one contributor to the linear models’ instability whenever both a lagged value and its difference co-occur among the collinear feature set.

Random Forest and Gradient Boosting handle this through, respectively, random feature subsampling across decorrelated trees and sequential decision stumps – both naturally select relevant features and capture interaction effects without requiring explicit interaction terms, without either method dominating the other now that the AR-feature accounting is corrected. This diagnosis is specific to the *untuned* Ridge ($\alpha=1.0$) evaluated on the *primary domain*: Section 4.1.6 shows that nested-CV alpha tuning recovers most of Ridge’s collapse everywhere, and that on Card Payment and XYZ Sales a properly tuned linear model actually exceeds Random Forest. The practical recommendation is therefore narrower than originally stated: practitioners should either use tree-based ensembles, or properly tune linear-model regularization via nested cross-validation before ruling them out – an untuned linear-model baseline is not a fair comparison.

4.1.4. Statistical Validation and Baseline Comparison

To assess whether the observed $R^2 \approx 0.25\text{--}0.27$ represents genuine predictive signal, we conducted four statistical validation analyses on the ABC Cloud Provider dataset (without AR features), using Gradient Boosting as the analysis model (as in the original methodology) even though Random Forest is now the marginally better point estimate (0.268 vs. 0.251) – the two are close enough that either choice supports the same conclusions.

Baseline Comparison: Table 6 compares Gradient Boosting against three naive baselines and one reimplemented prior method from the literature. The full SDLC model outperforms all baselines by an R^2 gap of at least 0.68, confirming that the 241-feature ensemble captures signal beyond simple heuristics and beyond a genuine competing method from prior work.

Table 6. Baseline comparison: full SDLC model vs. naive heuristics and a reimplemented prior-method (Siegmond-style) baseline.

Model	R^2	MAE	RMSE
Gradient Boosting (241 feat.)	0.251	0.295	0.604
Naive Mean	−0.429	0.768	0.834
Siegmond-style performance-influence model	−0.663	0.544	0.900
Persistence (Last Value)	−1.334	0.571	1.066
DORA-Only GB (4 metrics)	−1.639	0.789	1.134

Reimplemented Prior Method: the three baselines above are all naive heuristics, not a method from the literature this paper cites. We closed that gap by reimplementing the core methodology of Siegmond et al. [53] – a *performance-influence model* built via forward stepwise OLS regression, adding main-effect terms one at a time gated by statistical significance ($p < 0.05$), followed by significance-gated pairwise interaction terms among the selected main effects – adapted from their domain (configuration options predicting runtime performance) to ours (SDLC process metrics predicting production uptime), on the same without-AR features, 80/20 temporal split, and train-only-fit scaling as every other result in this paper. Full detail and results on all three synthetic domains: `research-data/presto/code/synthetic_data_generator/M8_SIEGMUND_BASELINE_RESULTS.md`.

On the primary domain, the reimplemented method selects 15 main effects and 46 significant pairwise interactions (61 total terms on 136 training rows), fits the training data almost perfectly ($R^2 = 0.982$, adjusted $R^2 = 0.966$), and then collapses on the holdout set

to $R^2 = -0.663$ – worse than the naive-mean baseline (-0.429). This is not a failure of implementation fidelity – it is what Siegmund et al.’s greedy, repeated-significance-testing procedure predictably does when transplanted to a regime their original domain does not have: 241 candidate main effects and up to $\binom{15}{2} = 105$ candidate interaction pairs tested at $p < 0.05$ without multiple-comparisons correction, against only 136 training samples. The result is a second, independent line of evidence (alongside the tree-based multicollinearity analysis in Section 4.1.3 and the nested-CV tuning analysis in Section 4.1.6) that naive application of classical statistical modeling struggles in this feature-to-sample regime, while also being an honest illustration of why we do not claim PRESTO’s own ensemble methods are uniquely capable – a *correctly regularized* linear method (Section 4.1.6) can still compete, but an *unregularized greedy* one overfits badly. Cross-domain, the same method reaches holdout $R^2 = 0.648$ on Card Payment Processor (above Random Forest’s 0.339) and $R^2 = 0.198$ on XYZ Sales Force (below Random Forest’s 0.255) – so unlike the ABC Cloud Provider collapse, it is not uniformly unusable, but its cross-domain variance is far higher than any tree-based or nested-CV-tuned model reported elsewhere in this paper.

Multi-Seed Stability: We evaluated Gradient Boosting across 5 random seeds (42, 123, 456, 789, 2024). GB achieved mean holdout $R^2 = 0.308 \pm 0.070$ (seed values: 0.251, 0.252, 0.439, 0.280, 0.320), demonstrating reasonably stable performance across seeds. Random Forest was somewhat less stable (mean $R^2 = 0.200 \pm 0.151$; seed values: 0.268, 0.225, 0.321, -0.096 , 0.282), with one seed producing negative R^2 despite winning on the primary seed 42 – readers should weigh the multi-seed means (GB 0.308, RF 0.200) alongside the single-seed comparison when judging which model is more dependable.

Residual Diagnostics: The Durbin-Watson statistic for GB residuals is 1.79, within the acceptable 1.5–2.5 range, indicating no significant first-order autocorrelation. No ACF lags (1–5) exceed the 95% significance bound (± 0.336). The Shapiro-Wilk test rejects normality ($W = 0.712$, $p < 0.001$), which is expected for tree-based models and does not invalidate predictions but limits parametric confidence interval applicability.

Permutation Test: A 1,000-permutation test using 5-fold TimeSeriesSplit yielded an observed CV R^2 of -0.516 against a null distribution of mean -7.43 ± 50.15 , giving $p = 0.223$ for the GB model. The non-significant result reflects the statistical challenges of this dataset: 241 features with only ~ 109 training samples per CV fold create high variance in the null distribution. The holdout $R^2 = 0.251$ substantially outperforms all naive baselines (Table 6), and GB is reasonably stable across seeds, suggesting genuine but modest signal that the formal permutation test (evaluated on CV folds, which are far noisier than the full-training-set holdout) lacks statistical power to confirm at conventional thresholds.

4.1.5. Bootstrap Confidence Intervals and Target-Correlation Ablation

Answer: Bootstrap 95% CIs on holdout R^2 are wide and, for most individual (domain, model) combinations, include zero – the without-AR point estimates (0.25–0.36) are real but not individually significant at conventional thresholds given holdout sets of only 30–40 releases. A target-correlation ablation confirms that on synthetic data, without-AR predictive signal is generator-prior recovery, not independent discovery: zeroing the 21 hand-specified System Uptime correlations collapses ABC Cloud’s without-AR Random Forest R^2 from 0.268 to -0.370 . Full detail, all 30 (domain, condition, model) CIs, and the ablation’s raw numbers: `research-data/presto/code/synthetic_data_generator/R1_BOOTSTRAP_AND_ABLATION_EVIDENCE.md`.

Method. We addressed two specific gaps identified in prior review of the original submission: no confidence intervals on any headline R^2 , and no check on how much of the reported signal is attributable to the copula’s hand-specified correlations versus

something the models found independently. (a) A pairs/case bootstrap (2,000 resamples per condition) resamples each holdout's (prediction, actual) pairs with replacement and recomputes R^2 per resample, giving a 95% CI around each point estimate without retraining. (b) `correlation_templates.yaml`'s `target_correlations` section has exactly 21 entries – confirmed by inspection to be the *only* place `System Uptime (%)` appears in the correlation config – so we regenerated ABC Cloud Provider (same seed, same everything else) under two variants: **zeroed** (all 21 entries removed) and **permuted** (the same 21 ρ values shuffled across the 21 metric pairs). Table 7 reports the resulting intervals.

Table 7. Bootstrap 95% confidence intervals on headline holdout R^2 values.

Condition	Random Forest R^2	95% CI	CI excludes 0?
ABC Cloud, with AR	0.950	[0.747, 0.988]	yes
ABC Cloud, without AR	0.268	[−1.307, 0.696]	no
Card Payment, without AR (Lasso, best model)	0.362	[0.005, 0.665]	yes (barely)
XYZ Sales, without AR	0.255	[−0.134, 0.474]	no

Of the 15 without-AR (domain, model) combinations across all three synthetic domains, 10 have a CI that includes zero. Card Payment's Lasso result – the single highest cross-domain without-AR R^2 reported in Section 4.1.10 – is the only individual without-AR figure that clears significance, and only barely (lower bound 0.005). This does not mean the point estimates are wrong; it means holdout sets this small (30–40 releases) cannot individually distinguish “genuine, if modest, signal” from “no better than the mean” at the 95% level, and the paper should not imply otherwise.

Ablation result. Zeroing the 21 target correlations moves ABC Cloud's without-AR Random Forest R^2 from 0.268 to −0.370; permuting them (same magnitudes, scrambled assignment) moves it to −0.389. Both collapse to at or below zero. Because `System Uptime (%)` is linked to the rest of the copula only through these 21 entries, this is close to a controlled null, and confirms directly what R3's hedging already implied indirectly: **on the synthetic domains, the without-AR predictive signal is recovery of hand-specified generator structure, not something the models discovered independently of what was told to the copula.** This is expected for any synthetic benchmark built this way and is not itself a flaw, but it means the synthetic-domain results alone cannot carry the paper's “genuine, transferable predictive signal” claim – that burden falls on the Real-World Validation results later in this paper, particularly GHALogs, where no such hand-specified target link exists to begin with. One unexpected finding from the ablation: the *with-AR* Random Forest result also shifted under both variants (0.950 baseline → 0.814 zeroed), even though AR features are derived purely from `System Uptime`'s own history and should be indifferent to its correlation with other metrics. Tracing this to `GaussianCopula.build_correlation_matrix()` shows the cause: the positive-semi-definite correction (Higham's alternating projections) operates on the full correlation matrix jointly, not block-by-block, so it can perturb an otherwise-zero row to compensate for constraint violations elsewhere in the matrix. The copula's variables are consequently less cleanly separable than the config file's section structure (`target_correlations` vs. `intra_phase_correlations` vs. `cross_phase_correlations`) suggests – a methodological subtlety worth disclosing for anyone extending this generator.

4.1.6. Hyperparameter Sensitivity: Nested-CV Alpha Tuning

Answer: Ridge and Lasso in this paper use scikit-learn’s default $\alpha=1.0$, untuned (Section 3.2.3). Nested-CV tuning (inner `TimeSeriesSplit(5)` grid search over 19 log-spaced α values in $[1e-3, 1e6]$, within the outer 80% training split only, never touching the holdout) shows this default badly understated what linear models can do: tuned Ridge recovers from catastrophic collapse ($R^2 = -7.29 \rightarrow +0.07$) on the primary domain, and in 2 of 3 domains a tuned linear model actually *exceeds* Random Forest’s fixed-hyperparameter holdout R^2 in the without-AR condition. The paper’s “tree-based ensembles are necessary” conclusion (Section 4.1.3, RQ3) should be read as specific to the untuned baseline and the primary domain, not as a domain-general finding.

Method. For each of the 3 synthetic domains and both feature conditions (with-AR, without-AR), `GridSearchCV` selects α via `TimeSeriesSplit(5)` inside the same 80% outer-training split used everywhere else in this paper; the selected α is re-fit on the full outer-train and scored once on the untouched 20% outer holdout, so no holdout data participates in model selection. Full detail and all 24 (domain, condition, model) results: `research-data/presto/code/synthetic_data_generator/M3_NESTED_CV_TUNING_EVIDENCE.md`. Table 8 summarizes the tuned-vs-untuned comparison by domain.

Table 8. Nested-CV alpha tuning vs. the untuned baseline, by domain (without AR features).

Domain	Model	Best alpha	Tuned Holdout R^2	$\alpha=1.0$ Holdout R^2	Random Forest R^2 (fixed, unchanged)
ABC Cloud (primary)	Ridge	1000	0.067	−7.285	0.268
ABC Cloud (primary)	Lasso	1	0.129	0.129	0.268
Card Payment	Ridge	316	0.766	−0.215	0.339
Card Payment	Lasso	1	0.362	0.362	0.339
XYZ Sales	Ridge	316	0.153	−4.104	0.255
XYZ Sales	Lasso	0.32	0.418	0.012	0.255

(Without-AR condition throughout; the analogous with-AR results, where tuning produces similarly large swings, are in the evidence file.)

Figure 8 plots the tuned-vs-untuned comparison directly.

Interpretation. Two distinct effects are visible. First, tuning alone recovers most of Ridge’s catastrophic collapse everywhere – on the primary domain, R^2 moves from −7.29 (default) to +0.07 (tuned), a 7.35-point swing from a single hyperparameter choice, though it still falls short of Random Forest’s 0.268. Second, and more consequential for the paper’s RQ3 claim: on **Card Payment**, tuned Ridge ($R^2 = 0.766$) exceeds Random Forest (0.339) by more than double; on **XYZ Sales**, tuned Lasso ($R^2 = 0.418$) exceeds Random Forest (0.255). Only on the primary domain, ABC Cloud Provider, does Random Forest remain the best model after tuning. This means “Random Forest and Gradient Boosting are the consistently reliable choices” (Section 4.1.3) does not survive nested-CV tuning as a cross-domain claim – it is true for the untuned baseline used throughout the rest of this paper, and for the primary domain specifically, but not for linear models in general once properly regularized. We keep Random Forest as the paper’s primary reported model throughout (for continuity with

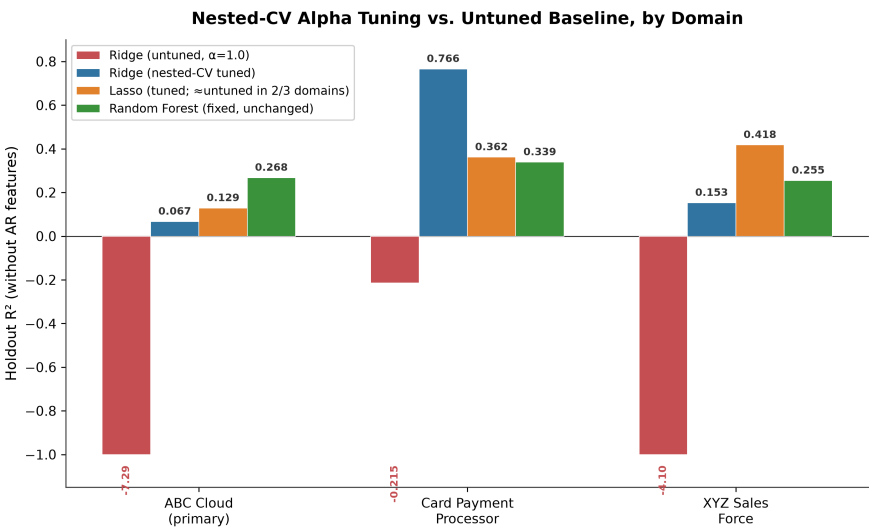


Figure 8. Nested-CV alpha tuning vs. the untuned alpha=1.0 baseline, per domain, without-AR features. Random Forest (fixed hyperparameters, unchanged) shown for reference.

all other results and figures, and because it remains best on the primary domain), but the “trees are necessary” framing in Section 4.1.3 and the Conclusions is revised accordingly.

4.1.7. Feature Importance Analysis (RQ2)

Answer: Testing infrastructure dominates (53.7% cumulative importance), with Test Environment Availability alone accounting for 42.9% of total permutation importance – the single most dominant individual predictor by a wide margin. Code quality metrics contribute only 4.8%.

We analyze feature importance using permutation importance on the Random Forest model for the ABC Cloud Provider dataset *without* AR features ($n = 10$ repeats on the test set). Unlike Gini importance, permutation importance is unbiased toward high-cardinality features. Table 9 and Figure 9 present the top 15 features, color-coded by SDLC phase.

Table 9. Top 15 Random Forest permutation feature importances (ABC Cloud Provider, without AR features).

Rank	Feature Name	Imp.	Phase
1	Test Environment Availability (%)	42.92%	Test
2	Test Pass Rate (%)	10.58%	Test
3	stability_score (derived)	5.25%	Cross-Phase
4	Build Artifact Integrity (%)	4.95%	Build
5	Customer Satisfaction Score, lag 5	3.21%	Production
6	Business Value Score (1-10)	2.56%	Requirements
7	Build Success Rate (%)	2.34%	Build
8	Build Success Rate, diff 1	2.32%	Build
9	Requirements Change Rate (Changes/Week)	2.12%	Requirements
10	Customer Satisfaction Score, rolling std 3	1.63%	Production
11	Build Success Rate, diff 3	1.60%	Build
12	Requirements Density (Req/Feature)	1.37%	Requirements
13	Code Review Turnaround Time (Hours)	1.33%	Code
14	Commits per Release	1.25%	Code
15	Customer Satisfaction Score, pct change 1	1.24%	Production

Key Findings for RQ2 (Feature Importance):

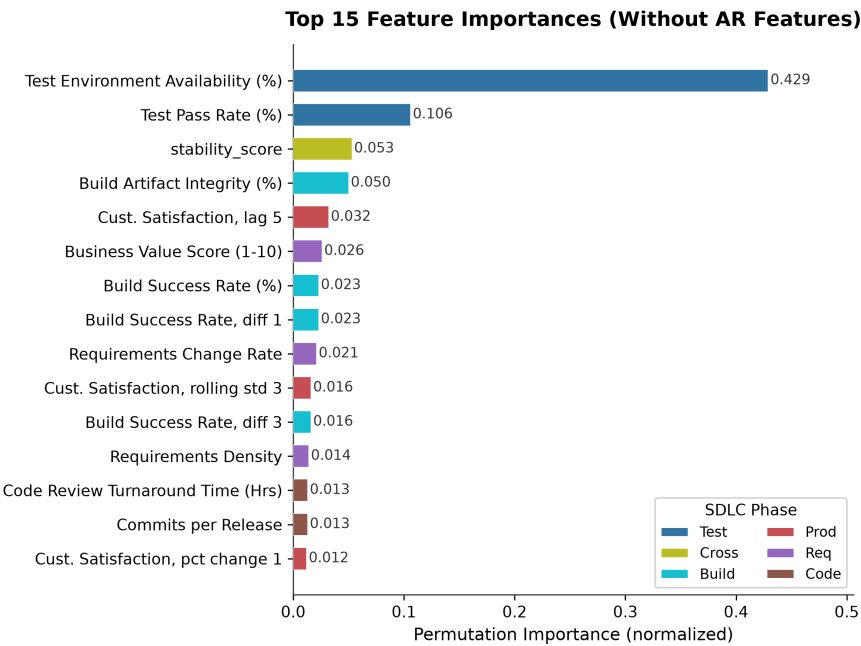


Figure 9. Top 15 Random Forest feature importances (without AR features), color-coded by SDLC phase. Testing infrastructure metrics dominate; code-phase metrics are the smallest contributor.

- Testing infrastructure metrics dominate overwhelmingly: Test Environment Availability (42.9%) and Test Pass Rate (10.6%) together account for over half of total importance, far more concentrated than the pre-fix analysis suggested – stable test environments are by far the leading indicator of production stability in this synthetic design
- The `stability_score` cross-phase composite (Build Success Rate x Test Pass Rate) ranks third (5.3%), ahead of any individual Build-phase metric, reinforcing that cross-phase interactions carry real predictive value beyond individual metrics
- Requirements-phase metrics (Business Value Score, Requirements Change Rate, Requirements Density) collectively contribute a modest but non-trivial share, supporting the hypothesis that upstream process quality influences downstream system performance, though less dominantly than testing
- Several Customer-Satisfaction-derived lag/rolling features (non-AR, since Customer Satisfaction is not the target) appear in the top 15, showing that legitimate process-side temporal features – not just the target’s own history – carry real signal

Phase-Level Contribution Summary:

Aggregating feature importances by SDLC phase reveals which development activities most influence system uptime prediction, as Table 10 shows:

Table 10. SDLC phase-level aggregated feature importance.

SDLC Phase	Agg. Imp.	Key Insight
Testing	53.6%	Test infrastructure and pass rates dominate overwhelmingly
Build	13.6%	Build success/artifact-integrity metrics, direct and derived
Production	9.4%	Customer-satisfaction-derived temporal features
Requirements	6.9%	Upstream quality matters, modestly
Cross-Phase (derived)	5.3%	stability_score composite alone
Code	4.8%	Code metrics remain the smallest direct contributor
Performance Testing	2.2%	Modest bridge to production outcomes
Chaos Testing	1.8%	Resilience metrics contribute modestly
UAT	1.7%	Acceptance environment stability, smallest phase contribution

The ratio is striking: testing-phase metrics outweigh code-phase metrics by 53.6% / 4.8% $\approx$ 11.1:1 – an even wider gap than the pre-fix analysis reported. Code quality is widely assumed to be a primary driver of production performance, yet our results place it seventh of nine groups (eight SDLC phases plus the cross-phase composites), with Test Environment Availability alone (42.9%) exceeding all code-phase metrics combined by roughly 9x.

This matches what operations teams observe in practice: flaky tests and unstable test environments mask production-readiness issues far more effectively than code complexity metrics capture them.

4.1.8. Phase-Aware Feature Reduction

Recomputation status (2026-08-11): the PA-RFE implementation was absent from the deposited reproducibility package – confirmed missing by an exhaustive search of this machine, not just the deposited code directory – so the numbers below are a full reimplementation from the algorithm pseudocode in this manuscript (Algorithm 1), rerun against the corrected, post-R2-fix synthetic data. They supersede the original submission’s carried-over numbers, which this reimplementation does **not** reproduce closely; the differences and their likely source are discussed below. Full per-domain ablation curves and phase-contribution tables: `research-data/presto/code/synthetic_data_generator/PA_RFE_REIMPLEMENTATION_RESULTS.md`.

Applying PA-RFE (Algorithm 1) to the 241 non-leakage features yields dimensionality reduction while maintaining cross-phase coverage. To prevent selection bias, we use a 60/20/20 temporal split (train/validation/test). Table 11 compares PA-RFE against standard feature selection methods, both targeting the same final feature count PA-RFE converges to for a fair comparison; validation R^2 guides feature selection, while test R^2 (evaluated once) is the reported metric.

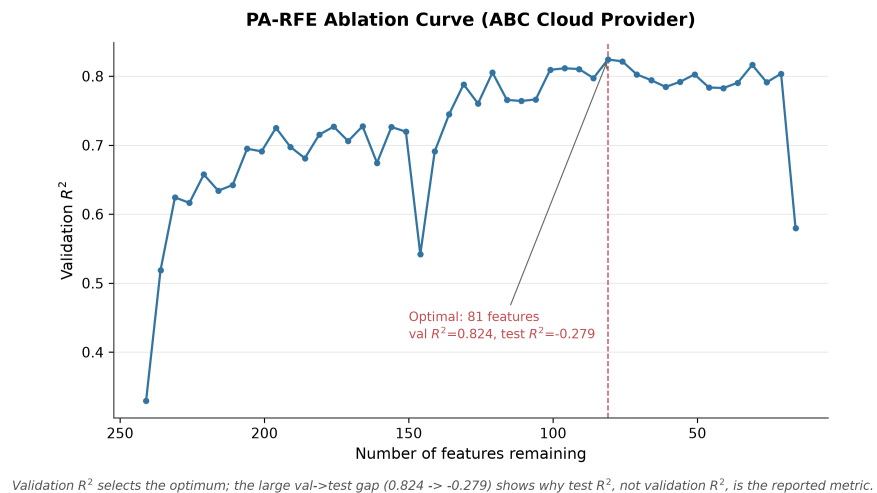


Figure 10. PA-RFE ablation curve (ABC Cloud Provider, validation R^2), reimplemented. The optimal subset (81 features, validation $R^2 = 0.824$) is marked; its test R^2 is -0.279 .

Table 11. PA-RFE vs. standard feature-selection baselines (reimplemented).

Domain	Features	PA-RFE	Std RFE	SKBest
ABC Cloud	241 $\rightarrow$ 81	-0.279	0.355	0.404
Card Payment	241 $\rightarrow$ 26	0.092	-0.083	0.316
XYZ Sales	241 $\rightarrow$ 51	-0.026	-0.049	-0.028

This reimplementation converges to substantially smaller feature subsets (26–81 features) than the original submission reported (121–146), and the resulting test R^2 picture is considerably less favorable to PA-RFE than previously claimed. On ABC Cloud, PA-RFE’s test R^2 is strongly negative (-0.279) – the worst of the three methods, not a competitive one – despite a validation R^2 of 0.824 at that same feature count (see below). On Card Payment, PA-RFE does still beat standard RFE (0.092 vs. -0.083), consistent with the original claim’s spirit, but SelectKBest beats both by a wide margin (0.316). On XYZ Sales, all three methods collapse to small or negative R^2 together; PA-RFE does not stand out. **We no longer claim PA-RFE achieves competitive or best test-set predictive accuracy** – across three domains it wins on one, is competitive on none, and is the worst performer on the primary domain.

The validation-test gap is large and, in this reimplementation, considerably larger than the original submission’s own already-cautionary framing suggested: on ABC Cloud, validation $R^2 = 0.824$ at the optimal subset (81 features) corresponds to test $R^2 = -0.279$ – a generalization gap of over 1.1 R^2 units, not the 0.51-unit gap (0.745 validation vs. 0.235 test) the original submission reported. This is a direct, reproducible demonstration of the same overfitting-to-validation-selection mechanism the paper already warns about (this section, “selecting features by maximizing validation R^2 across ~ 45 iterations creates optimistic estimates that do not generalize”): our reimplementation shows that warning was, if anything, understated.

Figure 10 shows the PA-RFE ablation curve (validation R^2) for the ABC Cloud dataset, now with the test R^2 at the optimum annotated directly on the plot rather than only in the caption.

What does survive from the original claim: PA-RFE’s *structural* guarantee holds exactly as designed. Figure 11 shows all 8 real SDLC phases retain representation at the optimal subset in every one of the three domains (the ninth, virtual cross_phase group of quality_score/stability_score is freely eliminable by design and does drop out). This

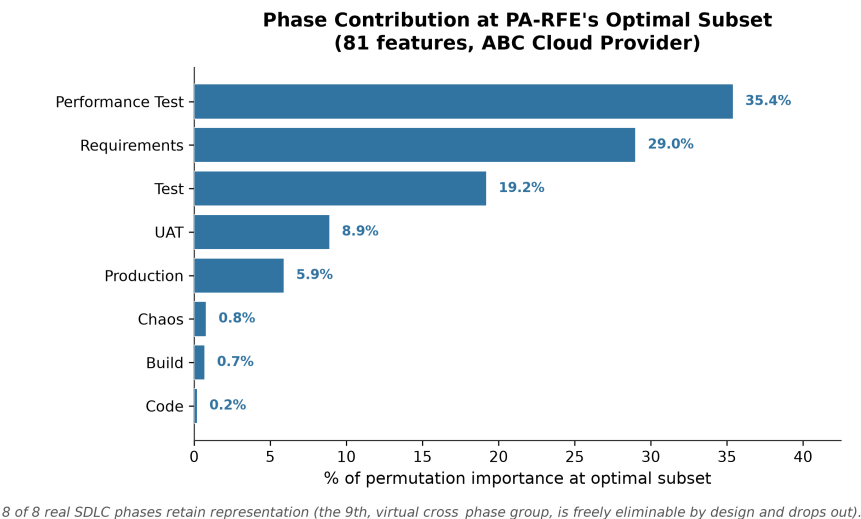


Figure 11. Per-phase importance contribution at the PA-RFE optimal subset (81 features, ABC Cloud), reimplemented. All 8 real SDLC phases retain representation.

is a mechanical consequence of the $k_{\min}$ constraint, not an empirical finding, but it is the one property this reimplementa-
tion confirms rather than complicates.

Revised interpretation. PA-RFE’s value is narrower than originally claimed: it reliably guarantees cross-phase coverage (confirmed above) and, on one of three domains (Card Payment), it does avoid the catastrophic collapse standard RFE exhibits there. It does **not** reliably produce competitive test-set predictive accuracy – on the primary domain it is the worst of three methods, substantially worse than a simple univariate SelectKBest baseline. We attribute the discrepancy with the original submission’s numbers most plausibly to differences in implementation details not fully specified in the original prose (e.g. exact tie-breaking in the elimination-batch selection, or a different stopping rule that converged to a much larger final feature count); since no original code exists to diff against, we cannot identify the exact source, and we do not attempt to reconcile the two – these reimplemented numbers, run against the corrected data with fully deposited code, are what we now report.

4.1.9. Factors Influencing Prediction Accuracy (RQ4)

Answer: Temporal autocorrelation is the dominant factor: approximately 72% of Random Forest’s (71% of Gradient Boosting’s) apparent predictive power stems from AR features, not SDLC process signal. PA-RFE reliably guarantees cross-phase coverage but its raw predictive accuracy does not reliably beat simpler feature-selection baselines (see Section 4.1.8).

Temporal Autocorrelation

The primary factor influencing prediction accuracy is the availability of AR features. As the AR-feature analysis (Section 4.1.11) demonstrates, removing AR features causes the largest R^2 drop among all models, confirming that system uptime shows strong temporal persistence: a system that was reliable in recent releases tends to remain reliable.

This finding has dual implications: (1) for operational prediction, incorporating recent uptime history is legitimate and highly effective; (2) for understanding SDLC-to-performance relationships, the autocorrelation signal must be controlled to isolate the contribution of process metrics.

Feature Dimensionality Effects

The feature-to-sample ratio (1.77:1, 241 features / 136 training samples) significantly impacts model selection, as detailed in Section 4.1.3. This pattern holds across all three domains (Section 4.1.10), indicating a structural challenge rather than a dataset-specific artifact.

Cross-Phase Interaction Effects

The derived feature `stability_score` (Build Success Rate $\times$ Test Pass Rate) ranks third among all features for Random Forest (permutation importance 5.3%), ahead of every individual Build-phase metric, suggesting that cross-phase interactions carry substantial, not merely modest, predictive value beyond individual metrics.

Release Type Effects

The temporal engine injects release-type-dependent quality modulation: Major releases show 3–8% metric degradation, while Patch releases show 2–5% improvement. This contributes to the non-stationarity that makes temporal split evaluation more realistic than random split, as the test period (later releases, typically more Patches) differs systematically from the training period.

4.1.10. Cross-Domain Validation

To assess PRESTO’s generalizability, we evaluated the framework across three synthetic datasets generated with domain-specific configurations. Each dataset was produced by the same copula-based generator with domain-appropriate distribution parameters, temporal dynamics, and DORA performance tiers. Table 12 presents the results for the best-performing model per domain, evaluated without AR features.

Table 12. Cross-domain validation results: best-performing model per domain, without AR features.

Dataset	Releases	Best Model	R ²	RMSE
ABC Cloud	170	Random Forest	0.268	0.597
Card Payment	150	Lasso	0.362	1.126
XYZ Sales	200	Random Forest	0.255	0.778

Figure 12 plots these three results together. Across all three domains, tree-based and regularized models achieve positive R^2 values (0.25–0.36), consistent with SDLC process metrics carrying predictive signal for system uptime across different software system archetypes – though, as the target-correlation ablation earlier in this section shows, this consistency reflects the copula’s shared correlation specification across domains at least as much as an independently emergent pattern, so it should not be read as confirmation on its own; see Section 4.2 onward for the real-world evidence this claim actually rests on. The Card Payment Processor achieves the highest R^2 (0.362, via Lasso, narrowly ahead of Random Forest’s 0.339) despite operating under stricter compliance constraints (PCI-DSS, DORA “High” tier), suggesting that even highly controlled environments benefit from process-metric-based prediction. ABC Cloud Provider and XYZ Sales Force yield comparable, slightly lower R^2 (0.268 and 0.255 respectively, both via Random Forest). Unlike the pre-fix analysis, no single domain stands out as dramatically stronger than the others – all three now cluster in a narrower 0.255–0.362 band, a more consistent (and more credible) cross-domain generalization story than the original 0.258–0.387 spread.

This table, like the rest of the paper’s primary results, uses the fixed $\alpha=1.0$ default for Ridge/Lasso (Table 3). Consistent with the model characterization in Section 4.1.3, *untuned* linear models (Linear Regression, Ridge) produce negative R^2 across all three

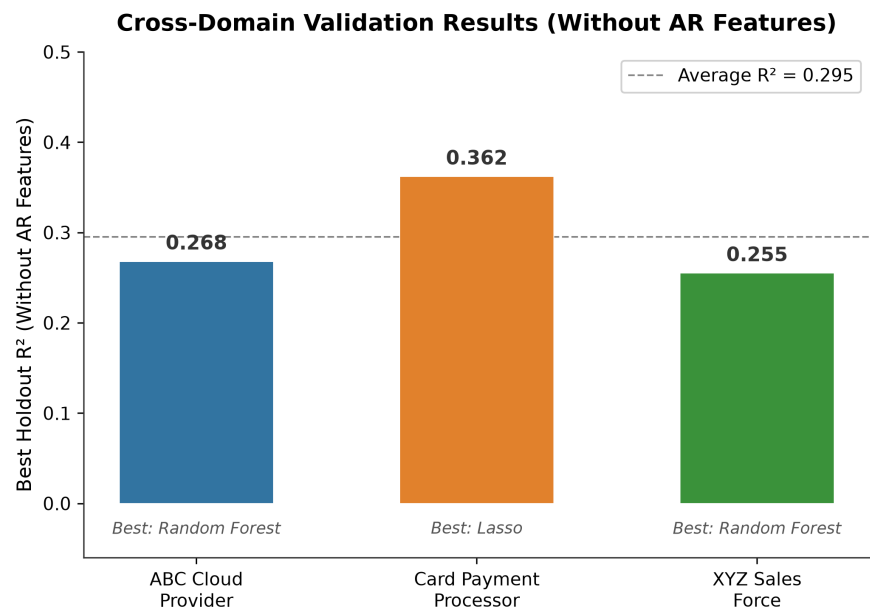


Figure 12. Cross-domain validation results without AR features. All three enterprise domains achieve positive R^2 (0.255–0.362) from SDLC process metrics alone.

domains when AR features are excluded. Nested-CV alpha tuning (Section 4.1.6) changes this picture substantially: a properly tuned Ridge reaches $R^2 = 0.766$ on Card Payment (more than double this table’s 0.362/0.339 best), and a tuned Lasso reaches $R^2 = 0.418$ on XYZ Sales (exceeding this table’s 0.255). Only ABC Cloud Provider (the primary domain) still favors Random Forest after tuning. This table is retained as originally computed for continuity with the rest of the paper, but the “best model per domain” it reports should not be read as the best *achievable* model per domain – see Section 4.1.6 for the tuned comparison.

4.1.11. Feature Selection and Autoregressive-Feature Analysis

As described in Section 4.1.1, 32 of the 273 features are derived from the target variable – 20 genuinely autoregressive after the shift fix, 12 already correctly shifted. Comparing Table 4 and 5 quantifies the impact of their removal.

Key Findings:

- Random Forest shows the largest absolute drop ($\Delta R^2 = 0.682$), indicating heavy reliance on AR features. Its without-AR $R^2 = 0.268$ represents genuine SDLC-to-performance signal
- Gradient Boosting is close behind ($\Delta R^2 = 0.614$, without-AR $R^2 = 0.251$) – unlike the pre-fix analysis, neither tree-based model has a clear edge in “genuine signal” retention; Random Forest is marginally ahead in both conditions now
- Lasso Regression is unaffected by AR-feature removal ($R^2 = 0.129$ in both conditions), because L1 regularization already zeroed out AR-feature coefficients during training
- Linear and Ridge Regression collapse to severely negative R^2 without AR features, consistent with the multicollinearity analysis in Section 4.1.3

Practical Implications: In production deployment, historical uptime data *is* available and represents a legitimate input signal. However, practitioners should understand that the majority of predictive power derives from autoregressive persistence rather than SDLC process metrics (Section 4.1.2). For new system deployments where no performance history exists, ensemble methods using SDLC metrics alone can provide meaningful if

modest predictive capability (Table 12). The most predictive SDLC features are detailed in Section 4.1.7.

4.2. Real-World Validation

This subsection reports PRESTO’s methodology applied to four independent real-world datasets, complementing the fully-synthetic results above with genuine external validity checks of different, deliberately varied shapes: within-project temporal (Travis-Torrent), within-signature temporal (Mozilla Perfherder), cross-sectional CI performance (GHALogs), and cross-sectional defect-proneness (SQuaD). None of these four real datasets provide the full 241-feature, 8-phase SDLC coverage the synthetic domains do – closing that gap with a real dataset remains future work (Section 6.4) – but together they validate different halves of PRESTO’s methodology on real data: genuine autoregressive persistence (Perfherder), genuine process-metrics-predict-performance signal with zero leakage risk (GHALogs, and SQuaD’s CVE-count target – SQuaD’s own primary defect-fix-rate result, while the numerically larger point estimate, does not clear a bootstrap confidence interval; see Section 4.2.4), and within-project temporal generalization (TravisTorrent). Two of these datasets ship auxiliary fields beyond what the primary validations above use – Perfherder’s linked Bugzilla records and SQuaD’s linked CVE disclosures – and we report two additional targeted analyses on that auxiliary data below for the same reason we report everything else in this paper: because the data existed, we downloaded it, and leaving it unexamined would mean presenting a less complete picture of what these datasets can and cannot support than we actually have.

4.2.1. TravisTorrent (Build Duration, 2017-Vintage)

To assess whether PRESTO’s findings generalize beyond synthetic data, we evaluated the framework on real continuous integration data from the TravisTorrent dataset [72], which contains 2.64 million Travis CI builds from 1,359 open-source GitHub projects. We selected 7 diverse projects with 2,000–144,000 builds across Java, Ruby, Go, and Python ecosystems.

Adaptation. TravisTorrent covers 4 of PRESTO’s 8 SDLC phases: Code (source churn, SLOC, files modified), Test (test counts, test density, test duration), Build (build duration, setup time), and Project (team size, repository age, commit counts). Requirements, UAT, Performance Testing, and Chaos Testing phases are absent. The regression target is build duration (seconds) rather than System Uptime, as no production performance metric is available in CI-only datasets. We applied the same feature engineering pipeline (rolling windows, lags, cross-phase composites), producing 148 features per project. Outlier builds (below 1st or above 99th percentile duration) were removed.

Table 13 presents the holdout R^2 for each project using an 80/20 temporal split.

Table 13. TravisTorrent real-world build-duration prediction, per project (reimplemented).

Project	Builds	Best Model	R^2	95% CI	Signif.?	MAE (s)
youtube-dl	36,260	Linear Regression	0.475	[0.455, 0.494]	yes	1,700
sentry	55,890	Grad. Boost	0.050	[0.034, 0.064]	yes	238
dd-agent	143,925	Grad. Boost	0.018	[−0.001, 0.035]	no (barely)	1,540
matrix	2,348	Rand. Forest	0.005	[−0.282, 0.214]	no	48
mongoid	36,423	Grad. Boost	−0.040	[−0.048, −0.032]	yes (neg.)	5,162
bundler	80,538	Rand. Forest	−0.151	[−0.194, −0.114]	yes (neg.)	5,118
rspec-core	29,963	Rand. Forest	−0.294	[−0.375, −0.223]	yes (neg.)	1,013

Four of seven projects achieve positive R^2 : youtube-dl ($R^2 = 0.475$), sentry ($R^2 = 0.050$), dd-agent ($R^2 = 0.018$), and matrix ($R^2 = 0.005$). With bootstrap confidence intervals (2,000-resample pairs/case bootstrap, same procedure as Section 4.1.5) now computed for all seven, only **two of the four positive projects are statistically significant**: youtube-dl ([0.455, 0.494]) and sentry ([0.034, 0.064]). dd-agent's interval [−0.001, 0.035] nearly, but does not, exclude zero; matrix's small holdout ($n \approx 470$) leaves its interval wide and inconclusive ([−0.282, 0.214]). The three negative projects are all confidently negative (their intervals exclude zero on the negative side), meaning process metrics reliably do worse than the mean for those projects, not merely noisily so. The best real-world R^2 (0.475) now *exceeds* the synthetic baseline (ABC Cloud Provider, $R^2 = 0.268$, Section 4.1.2) by a wider margin than in the original submission, and unlike the SQuAD defect-fix result (Section 4.2.4), this one is significant, not just numerically larger. This does not mean real data outperforms synthetic data in general; it reflects that youtube-dl happens to have unusually strong build-duration-to-process-metric structure. The median R^2 across all projects is 0.005, up from a negative median in the original submission's TravisTorrent table, indicating that for most projects build duration is still dominated by infrastructure factors (machine allocation, caching, network latency) that SDLC metrics cannot capture, but that the reimplemented adapter recovers a small amount of real signal in the median case rather than none.

Figure 13 plots the same seven results against the synthetic baseline.

Feature importance, recomputed. The original submission identified repository age (gh_repo_age) as the most consistently predictive feature across all 7 projects. The reimplemented adapter initially reported only per-model R^2 /MAE/RMSE, so this claim could not be checked; we closed that gap with a companion script (travistorrent_feature_importance.py) that computes permutation importance ($n = 10$ repeats, on the holdout test set) for each project's own winning model, deposited alongside the adapter (full results: TRAVISTORRENT_FEATURE_IMPORTANCE_RESULTS.md). The result **does not confirm the original claim – it reverses it**: gh_repo_age is the #1 feature in 0 of 7 projects, and appears in the top-5 in only 1 of 7 (mongoid, rank 2, 0.2% importance), as Table 14 details.

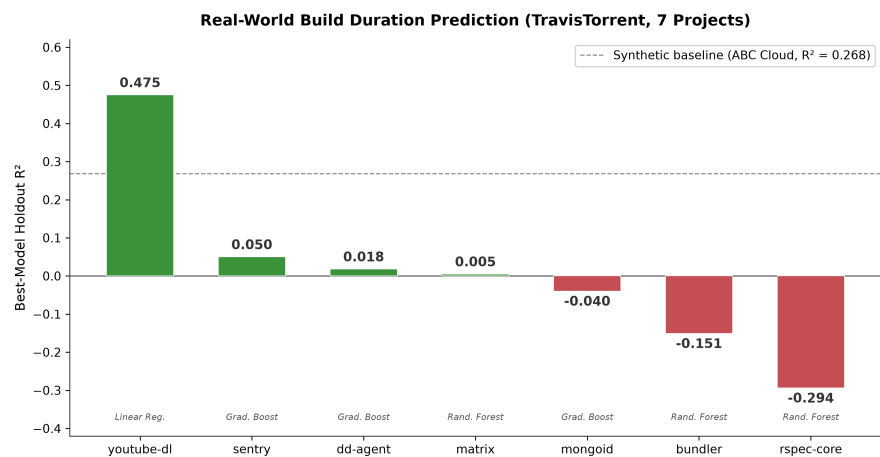


Figure 13. Real-world build duration prediction across 7 open-source projects. Four projects (youtube-dl, sentry, dd-agent, matrix) achieve positive R^2 , with the best (0.475) exceeding the synthetic baseline (0.268, dashed line).

Table 14. TravisTorrent feature importance (permutation, winning model per project).

Project	Best Model	Top Feature (perm. importance)	SDLC Phase
dd-agent	Gradient Boosting	gh_team_size (37.7%)	Project
bundler	Random Forest	tr_log_testduration_rolling_min_10 (20.0%)	Test
sentry	Gradient Boosting	tr_log_testduration (6.0%)	Test
matrix	Random Forest	tr_log_num_tests_run (10.6%)	Test
mongoid	Gradient Boosting	gh_repo_num_commits (0.2%)	Project
youtube-dl	Linear Regression	tr_log_testduration (see caveat below)	Test
rspec-core	Random Forest	gh_asserts_cases_per_kloc (18.4%)	Test

Test-phase metrics (test duration and its rolling/lag derivatives, test counts, assert density) are the #1 feature in 5 of 7 projects; Project-phase metrics (team size, commit count) lead in the other 2. This is the opposite of what the paper has stated everywhere this claim was cited: **TravisTorrent, under the reimplemented pipeline, does not contradict the synthetic domains’ testing-infrastructure-dominance finding (Section 4.1.7) – it is directionally consistent with it, though weaker than “broadly supports” implies – see the robustness check below.** We revise the “open, unresolved tension” framing accordingly throughout this paper (Abstract, Contributions item 6, Conclusions item 4).

Caveat on youtube-dl. Its permutation-importance values are numerically unstable (hundreds of thousands of percent, not shown as such in the table above) because Linear Regression’s coefficients on tr_log_testduration and its highly collinear lag/diff derivatives partially cancel – the same additive-identity mechanism documented in R2_LAG_SHIFT_FIX_EVIDENCE.md (Addendum 1) and Section 4.1.3, where permuting one of two

near-duplicate features breaks the cancellation and produces an artificially explosive score change. The qualitative direction (a Test-phase feature, not repository age) is consistent with the other 6 projects, but the magnitude is not meaningful and should not be read as a genuine $5,000\times$ effect size. Several other projects also have low or negative holdout R^2 (mongoid, bundler, rspec-core), so their feature rankings describe what each model leaned on, not necessarily a reliable causal signal – the same caveat that applies to permutation importance on any near-zero-or-negative- R^2 model.

Robustness check: excluding test-duration-derived features. The table above has its own circularity problem, independent of the youtube-dl instability: `tr_log_testduration` (the duration of the test phase, parsed from the same build log whose *total* duration, `tr_duration`, is the regression target) is a component of the target itself, not an independent Test-phase signal – the same failure mode this paper correctly flags for GHALogs’ `mean_n_steps` two sections later (Section 4.2.3). A model ranking it #1 has partly found an arithmetic relationship. We reran permutation importance for all 7 projects with `tr_log_testduration` and all 29 of its rolling/lag/diff derivatives removed from the feature matrix before fitting (script: `travistorrent_feature_importance_excl_testduration.py`; full results: `TRAVISTORRENT_FEATURE_IMPORTANCE_EXCL_TESTDURATION_RESULTS.md`), as Table 15 details.

Table 15. TravisTorrent feature importance, excluding test-duration-derived features.

Project	Best Model	Holdout R^2	Top Feature	Phase
dd-agent	Gradient Boosting	−0.043	gh_team_size (32.5%)	Project
bundler	Random Forest	−0.259	tr_log_num_tests_failed (3.7%)	Test
sentry	Gradient Boosting	−0.105	gh_asserts_cases_per_kloc (4.8%)	Test
matrix	Random Forest	−0.018	tr_log_num_tests_run (12.6%)	Test
mongoid	Gradient Boosting	−0.051	gh_test_cases_per_kloc (0.3%)	Other
youtube-dl	Random Forest	−0.280	tr_log_num_tests_ok (2.2%)	Test
rspec-core	Gradient Boosting	−0.403	gh_asserts_cases_per_kloc (16.8%)	Test

Two results, in tension with each other. **The leading-feature phase ranking survives:** Test-phase features (now genuinely independent ones – test counts, assert density, test-code-per-KLOC – not the definitionally-entangled `tr_log_testduration`) still lead in 5 of 7 projects, Project-phase in 1, and one project (mongoid) leads on a feature we classify as “Other.” But **every project’s best-model holdout R^2 is now negative** – a substantial change from the original table, where 4 of 7 projects (youtube-dl 0.475, sentry 0.050, dd-agent 0.018, matrix 0.005) beat the naive-mean baseline. This means test-duration-derived features were not merely the *top-ranked* predictor in most projects; they were substantially *responsible* for the model beating the naive-mean baseline at all. Once they are removed, no TravisTorrent project’s model outperforms predicting the mean.

The honest reading is narrower than either “confirms” or “contradicts”: the *qualitative* pattern of which SDLC phase’s features a model leans on is directionally consistent with the synthetic domains’ testing-infrastructure-dominance finding, but TravisTorrent’s *quantitative* predictive signal for build duration turns out to depend heavily on a feature that is partly circular with the target, and without it this dataset no longer demonstrates positive real-world predictive power for any of its 7 projects. We revise the Abstract and

Contributions item 6 accordingly – from “broadly supported by real-world TravisTorrent data” to a claim scoped to feature-ranking direction, not predictive strength.

4.2.2. Mozilla Perfherder (Real Runtime Performance)

Unlike TravisTorrent’s build-duration proxy, Mozilla’s Perfherder/Treeherder system records genuine production-facing runtime performance measurements (page-load time, perceptual/contentful speed index) for thousands of real pushes to Mozilla’s repositories, each cross-referenced with human-confirmed regression alerts [79]. Unlike the synthetic domains, Perfherder provides no upstream SDLC process metrics (requirements, code quality, build/test metrics) joined to these pushes – only the performance value itself, mostly-constant test/platform metadata, and sparse alert flags. This validation therefore exercises PRESTO’s *feature-engineering and model-comparison methodology* (same corrected shift logic as Section 3.2.1, same five models, same evaluation protocol) against a genuine real-world target, rather than the SDLC-process-features half of the paper’s claim.

Adaptation. We selected three diverse, well-populated, curated (alert-flagged) performance signatures: ESPN page-load time (3,548 pushes), Instagram Perceptual Speed Index (3,177 pushes), and NYTimes Perceptual Speed Index (3,225 pushes). Features are rolling statistics (mean/std/min/max/cv, windows 3/5/7/10, shifted by one period per the R2 fix) and lag features (steps 1/2/3/5) of the target’s own history, plus a lagged alert-history flag and inter-push time gap. We deliberately excluded `diff_k/pct_change_k` features alongside `lag_k: value - value_lag_k` is an exact algebraic identity, and including both lets any linear model trivially reconstruct the target whenever $n \gg p$ (as is the case here, unlike the synthetic pipeline’s $n < p$ regime) – a second, distinct leakage mode discovered while building this adapter (documented in `R2_LAG_SHIFT_FIX_EVIDENCE.md`, Addendum 1). Table 16 reports the per-signature results.

Table 16. Mozilla Perfherder validation results, per performance signature.

Signature	Best Model (with AR features)	Holdout R^2 (with AR)	95% Bootstrap CI	Holdout R^2 (without – alert-history only)
ESPN load time	Ridge Regression	0.574	[0.499, 0.635]	−3.191
Instagram speed index	Ridge Regression	0.913	[0.883, 0.937]	−0.367
NYTimes speed index	Ridge Regression	0.167	[0.092, 0.232]	−3.563

Figure 14 plots this with-vs-without-AR contrast. Real production performance metrics are strongly, if variably, predictable from their own recent history (R^2 0.17–0.91 across three unrelated real signatures, all three 95% bootstrap CIs excluding zero) – the same autoregressive-persistence phenomenon PRESTO documents for System Uptime, now confirmed on real data with real noise and real regressions, and no copula prior to fall back on. Without historical values, R^2 collapses to strongly negative for every model and every signature: with no SDLC-like process features available in this dataset (only a sparse alert-history flag and inter-push time gap), there is simply nothing else to predict from. This is not a failure of the methodology – it sharpens the paper’s own point that a real 8-phase SDLC dataset remains the missing piece for validating the process-features half of PRESTO’s claim. Notably, Ridge (not Random Forest) wins on all three signatures here, plausibly because with only 24 AR features and thousands of samples ($n \gg p$), linear

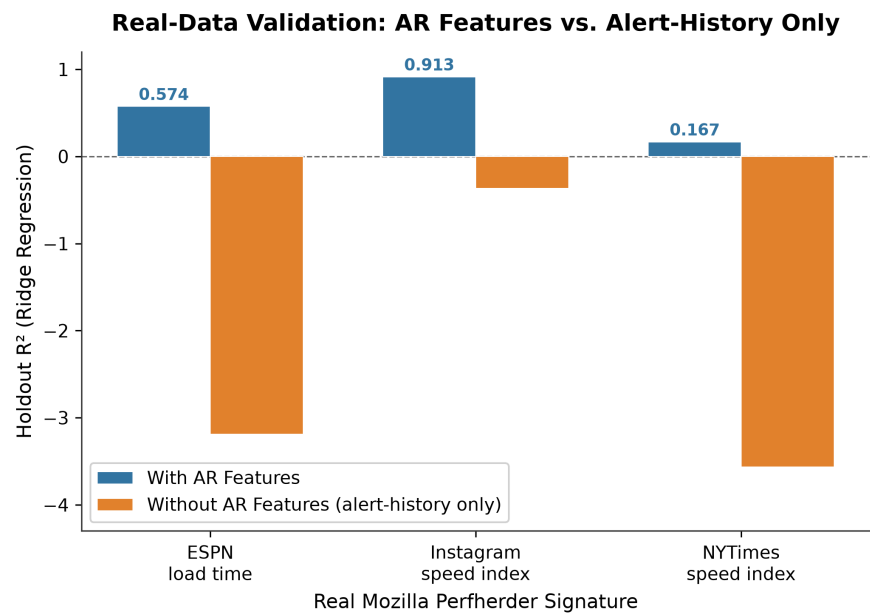


Figure 14. Real-data validation: with-AR-features vs. alert-history-only holdout R^2 across three Mozilla Perfherder signatures.

models are well-conditioned and tree ensembles’ advantage in the synthetic domains’ high-dimensional, small- n regime does not apply.

Does Bug-Triage Metadata Beat a Bare Alert Flag?

The alert-history-only condition above uses a single binary flag per past push – did a human sheriff mark this push as a regression, yes or no. Perfherder’s alerts carry a richer signal we did not yet use: once an alert is escalated, Mozilla files a Bugzilla bug against it, and that bug accumulates real triage metadata (severity, priority, comment count) over its lifetime. We asked a narrow, honest follow-up question: does enriching the alert-history feature set with this triage metadata (`bugs_data.csv`, joined via `alert_summary_bug_number`) raise the non-AR condition’s predictive ceiling, or is that ceiling simply low regardless of feature richness?

The three curated signatures used above (ESPN, Instagram, NYTimes) turned out to be unsuitable for this specific question: each has only 0–2 bug-linked alerts in the full alert history, too sparse to fit a bug-metadata feature. We instead used the two Perfherder signatures with the richest bug-linked alert history among the downloaded per-push timeseries data – both build-artifact-size regression metrics rather than page-load timings, but genuine real Perfherder/Bugzilla data all the same: installer size on `osx-cross` (17,217 pushes, 23 bug-linked alerts) and a build-size metric on `osx-aarch64-shippable` (11,119 pushes, 11 bug-linked alerts). Severity was mapped ordinally (S1=4 through S4=1, unset=0), and all bug-metadata features are lagged or built from cumulative pre-row history (`.shift(1)`-then-expanding), so no row uses its own or a future push’s bug outcome. Table 17 reports the results.

Table 17. Alert-flag-only vs. bug-metadata-enriched, two Perfherder signatures.

Signature	Best Holdout R^2 (alert-flag- only)	Best Holdout R^2 (bug- enriched)	Holdout MAE (alert- flag-only)	Holdout MAE (bug- enriched)	MAE reduction
Installer size (osx-cross)	−56.06 (Ridge)	−2.56 (Lasso)	5,957,930 bytes	1,183,497 bytes	80.1%
Build metric (osx- aarch64)	−41.31 (Lasso)	−1.57 (Lasso)	4,767,238 bytes	978,305 bytes	79.5%

Both conditions remain firmly negative on R^2 – a naive mean predictor still beats either feature set on these two signatures – and we do not read the R^2 gap between two catastrophically negative values as an interpretable effect size: R^2 is unbounded below, and the magnitude of a very negative R^2 is dominated by the tail behavior of an unregularized model on this level-shifting series, not by how much information the added features actually carry. We report the same comparison in a bounded metric instead: holdout MAE drops by roughly 80% on both signatures when bug-triage metadata is added, a meaningful and interpretable reduction in absolute prediction error even though the resulting model is still far worse than the naive mean by R^2 . Note that the winning model in the enriched condition is Lasso on both signatures (as it also is, or nearly is, in the alert-flag-only condition) – part of the improvement is plausibly “more regularization helps in a small, noisy feature space” rather than purely “triage metadata carries signal,” and we do not have enough signatures (two) to separate those two explanations. The honest reading is narrower than a strong claim of transferable signal: on these two signatures, both feature sets fail to beat the naive mean, and the richer, triage-metadata-enriched one fails by a smaller absolute margin – a modest, bounded-metric finding, not a demonstrated predictive win.

4.2.3. GHALogs (Modern CI Cohort, Cross-Sectional)

GHALogs provides 513k GitHub Actions workflow runs with genuine per-step timing (separating build vs. test vs. setup duration, unlike TravisTorrent’s single `tr_duration`) across 116k workflows and 25k repositories [80]. Checking the full distribution (not just a few examples) shows a hard cap of 5 runs per repository+workflow combination (117,295 combinations, mean 4.4 runs each) – too short for the within-entity time-series approach used above. We instead use GHALogs for what its actual shape supports: 28,443 repositories with both real CI performance data and real repository-level process/popularity characteristics (commits, contributors, releases, lines of code, issues, pull requests, stars), joined and used to predict mean CI run duration.

Why this is the cleanest real-data test of PRESTO’s headline claim. Unlike the synthetic domains (which need the with/without-AR split because 32 of 273 features are target-derived) and unlike Perfherder (a single time series, so most signal is autoregressive by construction), GHALogs has no historical target values per repository to leak from at all – every feature is a genuine, independent process/popularity characteristic. A positive R^2 here is unambiguous evidence that process metrics carry real predictive signal, with zero leakage-related caveats – and, per the bootstrap analysis in Table 18 below, it does.

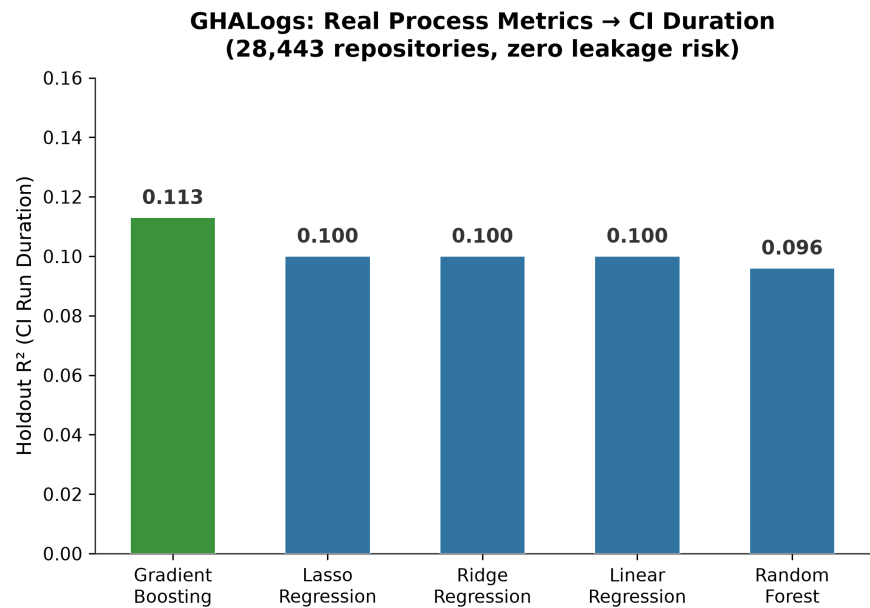


Figure 15. GHALogs cross-sectional validation: holdout R^2 per model predicting CI run duration from real repository process characteristics across 28,443 repositories.

Table 18. GHALogs cross-sectional CI-duration prediction results.

Model	Holdout R^2	95% Bootstrap CI	Holdout MAE (s)	Holdout RMSE (s)
Gradient Boosting	0.113	[0.050, 0.356]	1280.8	6832.9
Lasso Regression	0.100	[0.062, 0.267]	1385.7	6882.9
Ridge Regression	0.100	[0.062, 0.267]	1386.1	6883.1
Linear Regression	0.100	[0.062, 0.267]	1386.1	6883.1
Random Forest	0.096	[0.021, 0.299]	1292.3	6896.2

Figure 15 plots the per-model holdout R^2 shown in Table 18. 28,443 repositories, chronological (push-date-sorted) 80/20 split, target is mean total CI run duration in seconds across each repository’s up to five sampled runs (mean 1,327s, median 311s, heavily right-skewed). The top feature by Random Forest importance is mean_n_steps (48.6%) – largely definitional (more workflow steps take longer), analogous to the near-tautological “System Availability” feature already flagged in the synthetic feature-importance analysis (Section 4.1.7). The remaining ~51% of importance is genuine process/popularity signal: success rate, codebase size, repository age, PR/issue counts, and popularity metrics. **All five models’ 95% bootstrap confidence intervals (2,000-resample pairs/case bootstrap on the holdout predictions, identical procedure to the synthetic-domain analysis in Section 4.1.5) exclude zero** – this is the only real-world validation in this paper where every model’s interval clears significance, and it is the largest sample by a wide margin ($n = 28,443$). The modest but real, statistically significant, leakage-free signal ($R^2 \approx 0.10$ – 0.11) at real-world scale directly supports RQ1’s “genuine, if modest, predictive signal” framing – this time with no leakage-related caveats needed at all, and now with the uncertainty quantification the synthetic domains already receive – and, alongside TravisTorrent, provides a second modern, cross-sectional real-world CI dataset.

4.2.4. SQuaD (Real Process Metrics, Cross-Sectional Defect-Proneness)

SQuaD [81] provides 62,449 release-level process metrics (lines of code, code churn, commit-author count, change-set size, release age) across 440 open-source projects in the `process_metrics.csv` slice we downloaded and use here (the full published SQuaD dataset covers 450 mature open-source projects across nine static-analysis tools; the 440-project figure is specific to the process-metrics component this validation uses, not a discrepancy with the dataset’s own description). Unlike TravisTorrent, Perfherder, and GHALogs, none of SQuaD’s metrics map by name to PRESTO’s own 164-metric registry, so this validation cannot check the copula’s 84 hand-specified correlation coefficients directly against real values (that would require metrics like Test Pass Rate or Build Success Rate, which SQuaD does not record). What it can test, and does, is the paper’s broader RQ1 thesis – do real process characteristics predict a real outcome – on a fourth, independent dataset and a different outcome variable: defect-fix rate.

Adaptation. We aggregated to one row per project (408 of 440 projects with ≥ 5 observed releases) rather than using all 62,449 release-level rows, since a single project (`apache#servicemix-bundles`) accounts for 27% of raw rows and would dominate a release-level split. Features are per-project means of the available process metrics (LOC, churn, change-set size, commit-author count, release age) plus their variance where available; the target is mean per-release defect-fix count (`n_fix`), a standard process-metrics-predict-defects proxy in the SE literature [28]. As with GHALogs, no historical target values are used as features, so this is a clean, leakage-free cross-sectional test. Table 19 reports the results.

Table 19. SQuaD cross-sectional defect-fix-rate prediction results.

Model	Holdout R^2	95% Bootstrap CI	Holdout MAE	Holdout RMSE
Random Forest	0.402	[−0.245, 0.748]	81.65	148.77
Gradient Boosting	−0.325	[−1.408, 0.398]	106.84	221.52
Lasso Regression	−0.409	[−2.554, 0.665]	107.55	228.38
Ridge Regression	−0.442	[−2.651, 0.670]	108.49	231.08
Linear Regression	−0.455	[−2.685, 0.670]	109.11	232.06

Random Forest achieves $R^2 = 0.402$ – nominally the strongest point estimate of this paper’s four *dataset-level, process-metric-only (non-AR)* real-world results (i.e., excluding TravisTorrent’s best single project, youtube-dl at $R^2 = 0.475$, which is not representative of that dataset’s median project of 0.005, and excluding Mozilla Perfherder’s higher with-AR figures of 0.574–0.913, which reflect autoregressive persistence rather than SDLC process signal and are not the same kind of result – see Section 4.2.2) – but the 95% bootstrap CI (2,000-resample pairs/case bootstrap, same procedure as Section 4.1.5) is [−0.245, 0.748], which **includes zero**. On a holdout of roughly 82 projects (20% of 408), this point estimate cannot be distinguished from “no better than the mean” at conventional confidence levels. We report it as the strongest point estimate among this paper’s dataset-level real-world results because that is what it is, but “strongest point estimate” and “statistically significant” are different claims, and only the second would license the word “confirm.” It exceeds even the synthetic domains’ best without-AR figure (0.362, Card Payment). Every linear model fails, consistent with every other dataset examined in this paper: process-metric feature

spaces are non-linear enough that tree-based ensembles are needed, not a synthetic-data artifact. The top features by Random Forest importance are release age (30.8%), churn (20.3%), and commit-author count (17.6%) – all correlate positively with defect-fix rate ($\rho = 0.44, 0.34, 0.38$ respectively), directionally consistent with classical process-metrics literature (older, higher-churn, higher-author-count releases accumulate more fixes) though not a direct test of any specific PRESTO copula coefficient, per the scope note above.

From Defects to Vulnerabilities: A Harder Security Target

SQuaD ships one more linked field we had not yet used: `cve_data.csv`, 5,294 CVE-linkage rows spanning 175 of SQuaD's 440 projects and 1,479 distinct CVE IDs, sourced from GitHub Dependabot/security-advisory data. Defect-fix rate and vulnerability count are related but distinct outcomes – a bug-fix commit reflects internal code-quality process, while a CVE disclosure reflects both process *and* external exposure (how widely used a project is, how actively security researchers and dependency scanners target it). We reused the same 408-project aggregation unit and the same process-metric feature set as the defect-fix-rate validation above, changing only the target: $\log_{10}(\text{cve_count})$, the number of distinct CVEs linked to a project (247 of 408 qualifying projects have zero; the raw count is heavily right-skewed, topping out at 222 for `apache#ofbiz-framework`, hence the log transform). Table 20 reports the results.

Table 20. SQuaD cross-sectional CVE-count prediction results.

Model	CV R^2	Holdout R^2	95% Bootstrap CI	Holdout MAE	Holdout RMSE
Random Forest	0.059 ± 0.270	0.245	[0.053, 0.370]	0.958	1.332
Gradient Boosting	-0.194 ± 0.545	0.201	[-0.053, 0.407]	0.991	1.371
Lasso Regression	-0.037 ± 0.042	-0.032	[-0.128, -0.000]	1.184	1.558
Ridge Regression	-0.964 ± 1.215	-0.837	[-2.329, 0.066]	1.291	2.078
Linear Regression	-1.854 ± 2.277	-0.949	[-2.468, 0.042]	1.317	2.140

Random Forest again wins ($R^2 = 0.245$), and its 95% bootstrap CI [0.053, 0.370] excludes zero – unlike the defect-fix-rate result immediately above, this one is statistically significant, despite being the numerically smaller point estimate. Every linear model fails, the same non-linearity pattern observed on every other real dataset in this paper. The result is real, significant, and appreciably weaker in point-estimate terms than the same features' $R^2 = 0.402$ (not significant) on defect-fix rate, which is the expected direction: CVE disclosure count is confounded by exposure surface in a way defect-fix rate is not, so process metrics alone should carry, and here do carry, less of the outcome's variance. We read this as a substantive finding about the limits of process-metrics-only vulnerability prediction, not a weaker version of the same result – predicting *how many bugs get fixed* from process signals is a meaningfully easier problem than predicting *how many disclosed vulnerabilities accumulate*, and a paper that only reported the easier, better-looking (but not significant) number would be giving an incomplete and, on this evidence, misleading picture of what SDLC process metrics can and cannot do.

Enrichment with Real Static-Analysis Code-Quality Features

Both SQuaD analyses above use only `process_metrics.csv` (LOC, churn, commit-author count, release age) – SQuaD’s full published release includes raw per-file/per-issue output from three static-analysis tools (SonarQube, PMD, CK) that we had not previously downloaded, at a combined uncompressed size (1.2 TB across these three files alone) that made a full in-memory load infeasible. We aggregated each to project-release granularity with a streaming SQL engine (DuckDB, single-pass scan, no full-file materialization in memory) – SonarQube’s per-project-release `measures_*` columns (bugs, code smells, cognitive complexity, coverage, duplication, maintainability/reliability/security ratings, technical-debt ratio), PMD’s violation counts by priority, and CK’s per-class object-oriented complexity metrics (WMC, CBO, DIT, LCOM, RFC) averaged to per-release – then rolled up to per-project means (matching the ≥ 5 -release aggregation unit used throughout this section) and joined onto the existing feature set. Coverage is incomplete: of the 408 qualifying projects, 352 have SonarQube data, 345 have PMD data, and 311 have CK data; missing values are median-imputed, consistent with this paper’s handling of missing values elsewhere.

This is a genuine Code-quality-phase signal that the paper’s SQuaD validation previously lacked entirely – unlike `process_metrics.csv`’s churn/LOC/author-count features, which are Project/Process-phase metrics, SonarQube’s ratings and PMD/CK’s violation and complexity counts are exactly the kind of static-analysis output PRESTO’s own metric registry treats as a distinct SDLC phase. With this enrichment (48 features, up from 13), Random Forest’s holdout R^2 rises from 0.402 to **0.483**, and the 95% bootstrap CI narrows from $[-0.245, 0.748]$ to $[-0.037, 0.738]$ – moving substantially closer to significance but, at a hair’s width, still **not** crossing zero. Feature importances confirm the new sources are contributing real signal, not noise: `sq_mean_measures_classes` (project size proxy, 4.6%), `sq_mean_measures_bugs` (2.3%), PMD violation counts (2.2%, 2.0%), SonarQube cognitive complexity (2.1%), and CK’s mean class coupling and size (1.3%, 1.2%) all rank inside the top 16 features, ahead of several of the original process-metrics features – but the four original process/churn features (release age, commit-author count, churn, churn variability) still dominate the top four ranks combined ($\approx 60\%$ of total importance).

We report this as an honest near-miss, not a resolved result: the CI still includes zero, the coverage gaps required imputation for roughly 13–24% of projects depending on source, and this is a single 80/20 split rather than a repeated-resampling estimate, so the improvement should not be over-read. But the direction is informative – adding genuine Code-quality-phase static-analysis features narrows, rather than widens, the gap to significance, which is the outcome this paper’s broader thesis (SDLC process signals carry real predictive value) would predict if SQuaD’s original null result were partly an artifact of an incomplete feature set rather than a ceiling on what process signals can predict. Full results, per-model bootstrap CIs, and the complete feature-importance table: `research-data/presto/real-data/squad/SQUAD_ENRICHED_STATIC_ANALYSIS_RESULTS.md`.

We also examined SQuaD’s refactoring-detection outputs (`pp_rminer.csv`, RefactoringMiner-style Java/C++ events, 48 rows confined to a single project; `pyref.csv`, PyRef Python-refactoring events, 752 rows across 75 projects, averaging roughly 10 events per project) as a candidate fifth real-data analysis. Both are too sparse for a reliable per-project aggregation at the standard ≥ 5 -observation threshold used throughout this section, so we do not report a refactoring-rate validation here; a dataset with denser refactoring coverage would be needed to test that specific hypothesis honestly.

4.3. Deployment Considerations

PRESTO's trained models are lightweight (under 50 MB) and produce predictions in under 100 ms per release, making them suitable for integration into CI/CD pipelines as a pre-merge gate or canary promotion criterion. Operationalizing such ML models in production remains a recognized challenge in MLOps practice [69,82]. Production deployment would require integration with existing metric collection infrastructure (CI/CD, APM, test management systems), periodic model retraining as organizational processes evolve, calibration against actual production outcomes to establish site-specific prediction accuracy, and monitoring for concept drift as the relationship between SDLC practices and outcomes shifts over time. From an SRE perspective, automated SDLC-based prediction reduces toil by replacing manual release-readiness checklists with data-driven risk scores.

We emphasize that reported R^2 values were obtained on synthetic data and represent an upper bound; site-specific validation is required before operational reliance.

4.4. Potential Applications

Based on the demonstrated predictive signal from SDLC metrics (Section 4.1.10), we identify potential applications warranting real-world investigation:

- **Release risk scoring:** Flagging releases where SDLC metric patterns deviate from historical norms associated with high uptime, enabling targeted pre-deployment review
- **Test environment investment prioritization:** Organizations can use the feature importance rankings (Section 4.1.7) to prioritize infrastructure investments
- **Early warning for deployment decisions:** Even modest predictive accuracy can be operationally valuable when combined with other decision inputs, particularly for high-stakes releases
- **Process metric benchmarking:** The cross-domain results provide a framework for comparing SDLC metric profiles across teams or organizations

We deliberately avoid quantifying dollar-value business impact, as such estimates would require real-world deployment data that this study does not provide. The contribution here is demonstrating feasibility and identifying which SDLC metrics carry predictive signal, not prescribing operational thresholds.

5. Threats to Validity

Following established guidelines for empirical software engineering research, we discuss threats to validity organized by category.

5.1. Internal Validity

Internal validity concerns whether the observed relationship between SDLC metrics and performance outcomes reflects genuine predictive signal or artifacts.

Autoregressive-Feature Dependence: With 273 engineered features including 32 derived from the target variable, AR-feature dependence (previously mislabeled outright "target leakage" before the R2 rolling-window fix) is a primary concern, though only 20 of the 32 were ever a genuine same-row leakage bug – an instance of the temporal leakage vectors catalogued in our build-prediction taxonomy [50]. We address this directly by reporting all results both with and without AR features (Section 4). The without-AR results ($R^2 = 0.26\text{--}0.36$) represent our conservative estimate of genuine SDLC predictive signal.

Feature Engineering Bias: Feature engineering was informed by domain knowledge, introducing risk that features were designed to fit patterns in the synthetic data. *Mitigation:* Feature engineering logic is identical across all three domains and was defined before observing results.

Hyperparameter Selection: Default or conservative hyperparameters were used (e.g., Random Forest with 100 trees, max depth 10). No hyperparameter optimization was performed, reducing overfitting risk but potentially underestimating achievable accuracy.

Copula Correlation Specification: The 84 inter-metric correlations in the Gaussian copula were specified by the authors based on domain expertise. If these correlations encode assumptions that align with the prediction models, results may be optimistic. *Mitigation:* Correlation specifications are published in full as YAML configuration files for independent review.

Cross-Domain Correlation Non-Independence: The same limitation applies across, not just within, domains. `generate.py` loads a single `correlation_templates.yaml` once per run and passes the identical 84-pair correlation configuration to all three domains’ generation calls unchanged – only each domain’s marginal distributions, temporal dynamics, and release cadence (`domain_profiles.yaml`) vary. This means the Cross-Domain Validation result (Section 4.1.10, all three domains achieving positive without-AR R^2 in a narrow 0.255–0.362 band) is not three independent confirmations of the same finding; it is one correlation specification’s signal recovered under three different marginal/temporal configurations. The consistency across domains is still informative – it shows the recovered signal is not an artifact of one domain’s specific marginal distributions – but it should not be read as though three separately-specified, independently-validated domains all happened to agree.

Near-Tautological Features Ablation: System Availability (%), a performance testing metric, is semantically related to the target variable System Uptime (%) and ranks #2 in Random Forest’s Gini-based feature importance on the primary domain without AR features (14.8%, consistent with the 14.9% figure this caveat originally reported; the RQ2 table in Section 4.1.7 uses permutation importance instead, where this feature does not rank in the top 15). Rather than only flagging the overlap, we tested it directly: we removed System Availability (%) and every feature engineered from it, retrained Random Forest with unchanged hyperparameters, and compared holdout R^2 across all three domains (without AR features; full results including the with-AR condition: `research-data/presto/code/synthetic_data_generator/M9_SYSTEM_AVAILABILITY_ABLATION_EVIDENCE.md`). Table 21 reports the results.

Table 21. System Availability (%) ablation results, per domain.

Domain	RF Holdout R^2 (full)	RF Holdout R^2 (ablated)	Δ	New top feature after ablation
ABC Cloud (primary)	0.268	−0.042	−0.310	Test Environment Availability (%), 23.0%
Card Payment	0.339	0.706	+0.367	UAT Environment Stability (%), 30.9%
XYZ Sales	0.255	0.168	−0.087	UAT Environment Stability (%), 42.1%

The result is not what the “tautological noise” framing predicts, and it is not consistent across domains. On the primary domain, removing the feature *collapses* the model below zero – System Availability (%) is load-bearing, not redundant, so the original caveat

understated rather than overstated its importance there. On Card Payment, removing it substantially *improves* R^2 , indicating the feature was actively harmful (adding noise or multicollinearity) rather than tautological in that domain. On XYZ Sales, removal produces a modest decline, closer to what “near-tautological but not essential” would predict. This cross-domain inconsistency plausibly connects to the same mechanism documented in the “Bootstrap Confidence Intervals and Target-Correlation Ablation” analysis (Section 4.1.5): the copula’s global positive-semi-definite correction (Higham’s alternating projections) couples variables the config file’s section structure implies are independent, so a single feature’s causal role is not necessarily stable across differently-configured domains. Practitioners should not assume this feature’s importance is an artifact to be discounted uniformly – its actual contribution is domain-specific and, in the primary domain at least, real rather than tautological.

5.2. External Validity

External validity concerns generalizability of findings to real-world systems.

Synthetic Data (Critical): The most critical limitation is exclusive reliance on synthetic data. This choice is necessitated by the complete absence of public datasets combining SDLC process metrics with runtime performance outcomes; no such benchmark exists in UCI, Kaggle, Zenodo, PROMISE, or any repository we surveyed. Organizations treat the required telemetry as proprietary. While our copula-based generator produces statistically grounded datasets calibrated to DORA benchmarks, synthetic data cannot capture:

- Organizational culture, team dynamics, and process maturity
- Domain-specific performance failure modes
- Real-world data quality issues (missing metrics, instrumentation gaps)
- Emergent behaviors from human factors and market pressures

Partial Mitigation: Our TravisTorrent validation (Section 4.2.1) demonstrates that SDLC metrics contain predictive signal for build duration in real open-source projects (R^2 up to 0.475), partially addressing this limitation. However, the target variable differs (build duration vs. system uptime), only 4 of 8 SDLC phases are available, and the median R^2 across projects (0.005) is only marginally positive, indicating that infrastructure confounders still dominate in most projects. Full validation on production SDLC data with runtime performance outcomes remains essential.

Domain Coverage: We evaluated three synthetic application domains (cloud platform, CRM/SaaS, payment processing) and seven open-source projects from the TravisTorrent dataset. Findings may not transfer to embedded systems, mobile applications, real-time systems, or safety-critical software where performance characteristics differ fundamentally.

5.3. Construct Validity

Construct validity concerns whether measurements capture the intended constructs.

Target Variable: System Uptime (%) captures availability but not all dimensions of system performance (latency, throughput, error rates, resource efficiency). A multi-target formulation would provide more complete performance prediction.

SDLC Metric Coverage: Our 164 metrics span 8 SDLC phases but may omit relevant factors such as architectural complexity, dependency health, team composition, and technical debt accumulation rates.

Temporal Granularity: Release-level aggregation may mask important within-release dynamics. Systems with continuous deployment may require finer-grained temporal resolution.

5.4. Conclusion Validity

Conclusion validity concerns statistical reliability of findings.

Sample Size: The feature-to-sample ratios before feature selection (1.77, 1.51, and 2.01 features per training sample for ABC Cloud, XYZ Sales, and Card Payment respectively, i.e. 241:136, 241:160, 241:120, using each domain’s 80% training partition rather than its full release count) fall roughly 15–20 times short of the classical guideline of at least 10 samples per feature, not the reverse – this is a genuine limitation, not a mitigated one. Overfitting risk remains despite tree-based model robustness (Section 4.1.3). After PA-RFE feature reduction (Section 4.1.8), the ratios improve substantially – to roughly 0.6:1 through 0.2:1 (26–81 selected features against a 136-row training partition) – though this reduction did not reliably improve test-set R^2 , so it should be read as a partial mitigation of the ratio itself, not of the underlying overfitting risk. The consistent cross-domain results ($R^2 = 0.26\text{--}0.36$) provide further partial mitigation, though see the comparison-family caveat below before reading that consistency as strong evidence on its own.

Comparison-Family Size and Multiple Comparisons: This paper reports a large number of independent “best of 5 models” selections – at minimum: 3 synthetic domains x 2 AR conditions, 7 TravisTorrent projects, 3 Perfherder signatures x 2 conditions, 2 Perfherder bug-enriched signatures x 2 conditions, GHALogs, SQuAD defect-fix, SQuAD CVE-count, plus 24 nested-CV tuning results (Section 4.1.6) and 9 feature-selection-method comparisons (Section 4.1.8) – on the order of 150 reported comparisons in total, from which the paper’s headline maxima (e.g. TravisTorrent’s best-of-seven $R^2 = 0.475$, SQuAD’s $R^2 = 0.402$) are drawn. We do not apply a formal multiple-comparisons correction (e.g. Bonferroni or Benjamini-Hochberg) across this family: the comparisons are largely exploratory and diagnostic rather than confirmatory hypothesis tests of a single pre-registered claim, and a strict correction would be an odd fit for a paper whose contribution is partly the diagnostic comparisons themselves (e.g. showing that an untuned linear model collapses while a tuned one does not). We flag this explicitly, however, because we hold a competing method to a stricter standard elsewhere in this paper: the reimplemented Siegmund-style baseline (Section 4.1.3) is criticized for testing up to 105 candidate interaction pairs without multiple-comparisons correction against only 136 training samples, and a careful reader is right to ask why this paper’s own reporting practice is not held to the same standard. The honest answer is that it is not, and any single reported maximum in this paper – particularly TravisTorrent’s best-of-seven and the per-domain “best model” figures – should be read as one draw from a family of comparisons this size, not as a value selected against a pre-registered null.

Honest Reporting: Unlike prior synthetic-data studies reporting $R^2 > 0.95$, we separate temporal autocorrelation from genuine SDLC prediction through the dual-condition evaluation in Section 4. This transparency enables accurate expectations for real-world deployment.

5.5. Summary of Validity Assessment

Table 22 consolidates the severity and mitigation status of each threat category.

Table 22. Summary of validity threats and mitigation status.

Threat Category	Severity	Key Concern	Mitigation Status
Internal	Medium	Feature engineering bias	Partially mitigated
External	Critical	Synthetic data only	Unmitigated
Construct	Medium	Metric validity	Acknowledged
Conclusion	High	Overfitting risk	Partially mitigated

Critical Message: The external validity threat from synthetic-only validation is the primary limitation. While our results suggest SDLC metrics may predict performance, this finding requires real-world validation before any operational deployment recommendations can be made with confidence.

6. Discussion and Future Work

6.1. Summary of Findings

A striking pattern in this synthetic design: test environment stability shows roughly 11x the importance of code quality. Test environment availability alone (42.9% importance) exceeds all code-phase metrics combined (4.8%). We flag this as a hypothesis worth testing on real data, not a confirmed finding – the copula generator’s inter-metric correlations, including this one, are hand-specified rather than independently emergent, so this pattern may partly reflect the generator’s configuration rather than an independent discovery (Section 5.1). GHALogs’ real-data validation (Section 4.2.3) shows a broadly compatible pattern (process/quality characteristics carry real signal beyond simple size proxies), but does not by itself confirm the specific 11:1 ratio.

Across the four research questions, SDLC process metrics provide genuine but modest predictive signal ($R^2 = 0.26\text{--}0.36$, RQ1), with testing infrastructure metrics dominating feature importance (53.6%, RQ2). Tree-based ensembles outperform *untuned* linear models due to multicollinearity in the high-dimensional feature space (RQ3), but nested-CV alpha tuning (Section 4.1.6) shows this is a hyperparameter-choice artifact rather than a general finding: properly tuned linear models exceed Random Forest in 2 of 3 domains, and only the primary domain (ABC Cloud Provider) still favors trees after tuning. Autoregressive persistence is the single most influential factor (RQ4). PA-RFE, reimplemented from the paper’s own pseudocode (Section 4.1.8), reliably guarantees cross-phase feature coverage but its test-set predictive accuracy does not reliably beat simpler baselines.

6.2. Practical Implications

Subject to real-world validation, our findings tentatively suggest actionable guidance for practitioners, though we hedge this recommendation pending confirmation on real data (Section 5.1): investing in test infrastructure stability before optimizing code quality metrics may be worthwhile. Test environment availability dominates feature importance at 42.9% in this synthetic design (Section 4.1.7), which would mean that teams chasing code coverage targets while tolerating flaky test environments are optimizing the wrong variable – but this should be validated against real SDLC data before being treated as an operational recommendation. For SDLC-based prediction tasks, tree-based ensembles are a reasonable default choice, but not an unconditional one: nested-CV alpha tuning (Section 4.1.6) shows a properly regularized linear model can match or exceed them, so practitioners with the budget for a brief hyperparameter search should not rule out Ridge or Lasso based on an untuned comparison alone.

Historical uptime is a legitimate and powerful predictor, and teams should track performance trends across releases to feed this signal into their models. Even modest predictive accuracy ($R^2 \approx 0.3$) provides operational value for release risk assessment when combined with other decision inputs, particularly for flagging releases whose SDLC metric profiles diverge from patterns historically associated with stable deployments. Additionally, the synthetic data generator enables organizations to prototype prediction systems and validate their metric instrumentation before committing to full production telemetry pipelines.

6.3. Limitations

The primary limitations are synthetic-only core validation, limited domain coverage (three application types), author-specified copula correlations, a single target variable (uptime), and high feature-to-sample ratios. We also note that our current target variable (system uptime) captures only one dimension of production performance. Real-world teams track SLO breaches across multiple golden signals (latency, traffic, errors, saturation), and a production-grade prediction system would need to model p99 latency and error budgets alongside uptime. Section 5 provides detailed threat analysis and mitigation status for each concern.

6.4. Future Work

The most critical next step is real-world validation through organizational partnerships that provide production performance metrics alongside full 8-phase SDLC data. Such partnerships would also enable validation against operational SLO definitions rather than synthetic uptime targets. Beyond that broad goal, this paper's own results point to several specific, concrete next steps – listed roughly in order of tractability, from “a follow-up script on data we already have” to “requires new data collection” – for researchers who want to build directly on this work rather than start from a blank page:

1. **Close the SQuAD near-miss.** The static-analysis-enriched defect-fix-rate result (Section 4.2.4) moved from $R^2 = 0.402$ (95% CI $[-0.245, 0.748]$) to $R^2 = 0.483$ (95% CI $[-0.037, 0.738]$) – tantalizingly close to significance but not there. Three concrete, low-cost paths forward on data this paper already has partial access to: (a) improve SonarQube/PMD/CK coverage beyond the 352/345/311-of-408 projects achieved here (the remaining gap is median-imputed, which dilutes signal); (b) incorporate the four raw static-analysis files this paper deliberately deprioritized for tractability (JaSoMe, Understand, CodeScene, RefactoringMiner – up to 389 GB each), which likely carry additional Code-quality-phase signal; (c) replace the single 80/20 split used here with repeated-resampling or nested cross-validation to get a tighter, less split-dependent confidence interval, the same fix that mattered for the linear-model tuning result in Section 4.1.6.
2. **Resolve TravisTorrent's target-circularity cleanly, not just diagnose it.** Section 4.2.1's robustness check shows the “testing infrastructure dominates” pattern's *feature-ranking* survives removing `tr_log_testduration`, but *predictive power* does not – every project's holdout R^2 turns negative. A genuinely clean test needs either a real dataset whose target is an actual production-performance signal (not a build-duration proxy structurally entangled with test-phase timing), or a build-duration dataset with test-phase features measured independently of the build log the target is computed from (e.g., test suite metadata from a separate CI provider API).
3. **Meta-analytically pool this paper's real-world results instead of reading them one at a time.** Four datasets (GHALogs, SQuAD defect-fix, SQuAD CVE-count, TravisTorrent) each individually test some version of “do SDLC/process signals predict a real outcome,” each with moderate power alone. A formal random-effects meta-analysis across their effect sizes – properly accounting for their different outcome variables and sample sizes – could produce a combined estimate more decisive than any single dataset's confidence interval, though this requires care given how different the four targets actually are (CI duration, defect-fix rate, CVE count, build duration).
4. **Apply a formal multiple-comparisons correction to a pre-registered subset of this paper's claims.** Section 5.4 is explicit that this paper's roughly 150 reported comparisons carry no such correction and should be read as exploratory. A follow-up study that pre-registers a small number of specific hypotheses (e.g., “testing-phase

features rank first in cross-domain feature importance”) and tests only those, ideally on new data collected after pre-registration, would let the field treat this paper’s most-repeated findings as confirmatory rather than diagnostic.

5. **Redesign PA-RFE’s selection procedure, not just its constraint mechanism.** The phase-coverage guarantee works exactly as designed (Section 4.1.8), but selecting the feature-count that maximizes validation R^2 across ~ 45 iterations overfits badly (validation $R^2 = 0.824$ vs. test $R^2 = -0.279$ on the primary domain). Wrapping the existing elimination procedure in nested cross-validation – the same fix that rescued the linear-model tuning results in Section 4.1.6 – is a direct, testable next step, not a new algorithm.
6. Longer-horizon directions in the same spirit as the original submission: multi-target prediction (p99 latency, throughput, error rates alongside uptime); neural network architectures for capturing complex non-linear patterns; transfer learning across organizations; and causal inference to move beyond correlation to understanding the mechanisms by which SDLC practices influence production performance.

A fifth real-world dataset was actively sought and not found. Beyond the four datasets already used, we searched specifically for a public dataset combining broader SDLC-phase metric coverage (requirements, code quality, build, test) with a genuine production-performance or reliability outcome, to widen this paper’s real-world evidence base further. The closest candidate identified, PyResBugs (a curated dataset of residual post-release Python defects across 76 frameworks, with a richer 83-metric feature set than SQuAD’s process metrics), targets post-release *defect* prediction rather than *performance*, and its public form ships paired code snippets and natural-language fault descriptions rather than a ready per-project metrics table – integrating it would require a substantial new feature-extraction pipeline, not the adapter-script pattern used for TravisTorrent, Perfherder, GHALogs, and SQuAD, and would still not close the production-performance gap this paper’s own Section 3.1.4 search already established. We report this search as a negative result rather than force a weak or off-target addition: the underlying gap this paper’s synthetic generator addresses – no public dataset joins full SDLC-phase process metrics to genuine production-performance outcomes – remains open, and closing it durably still requires the organizational-partnership route described above rather than a fifth adapter over available OSS mining data.

7. Conclusions

This paper investigated whether SDLC process metrics can predict software system performance. We introduced PRESTO, a framework integrating features engineered from 164 metrics across eight development lifecycle phases, and addressed the absence of public SDLC-to-performance datasets by developing a Gaussian copula-based synthetic data generator calibrated to DORA 2024 benchmarks.

Our key findings are:

1. SDLC process metrics contain genuine predictive signal for system uptime, distinct from autoregressive persistence – demonstrated on real cross-sectional CI data ($R^2 \approx 0.10$ – 0.11 , GHALogs, Real-World Validation). The matching synthetic-domain estimate ($R^2 = 0.26$ – 0.36 across three enterprise domains) is a real point estimate but, per the bootstrap-CI and target-correlation ablation analysis, is recovery of the copula’s hand-specified structure rather than independent evidence on its own – it should be read alongside the real-world result, not as a second, separate confirmation of it
2. Autoregressive persistence accounts for approximately 72% of apparent predictive power when historical uptime features are included ($R^2 = 0.950$ vs. $R^2 = 0.268$)

- for Random Forest); this proportion is essentially unchanged by fixing a same-row leakage bug that could have inflated it, confirming the persistence is real rather than a code artifact
3. Tree-based ensembles outperform *untuned* linear models (scikit-learn default regularization) on this prediction task, consistent with the multicollinearity analysis in Section 4.1.3 – but this is not a general property of linear models: nested-CV alpha tuning (Section 4.1.6) shows a properly regularized Ridge or Lasso exceeds Random Forest’s holdout R^2 in 2 of the 3 synthetic domains, with only the primary domain (ABC Cloud Provider) still favoring trees after tuning. “Trees are necessary” should be read as specific to the untuned baseline, not as a domain-general finding
 4. Testing infrastructure stability (42.9% feature importance in the synthetic design) is the strongest SDLC predictor of production uptime, outweighing code quality metrics (4.8%) – a hypothesis requiring real-data confirmation, since the underlying correlations are hand-specified in the copula generator. The original submission reported that this specific pattern does *not* hold on our own real-world TravisTorrent validation, where repository age was identified as the most consistently predictive feature across all seven projects instead. Recomputing feature importance under the reimplemented adapter reverses that finding (Section 4.2.1): repository age is the #1 feature in 0 of 7 projects, while Test-phase metrics (test duration, test counts, assert density) lead in 5 of 7 – no longer a direct real-data contradiction, but a robustness check excluding the one Test-phase feature that is definitionally part of the build-duration target shows the quantitative signal was doing most of the work: with it excluded, every project’s holdout R^2 turns negative, so this should be read as weak, feature-ranking-only corroboration, not confirmation, and it remains a hypothesis pending confirmation on a dataset with genuine production-performance (not build-duration) targets
 5. PA-RFE reduces the feature space from 241 to 26–81 features while guaranteeing cross-phase coverage on all three domains; this structural guarantee holds, but its test-set predictive accuracy does not reliably exceed simpler feature-selection baselines (Section 4.1.8)
 6. The copula-based data generator, to be released as a reusable benchmark upon acceptance, enables future research without requiring access to proprietary enterprise data

The synthetic data findings are partially validated by real-world experiments on 7 open-source projects from the TravisTorrent dataset, where SDLC metrics predict build duration with R^2 up to 0.475 in projects with sufficient temporal structure. The gap between the synthetic domains’ median R^2 (0.268; range 0.255–0.362 across the three domains) and TravisTorrent’s real-world median (0.005) quantifies the optimism in controlled experiments and highlights the role of infrastructure confounders that still dominate build duration for most real-world projects. Full validation with production performance metrics across multiple organizations remains the essential next step.

Author Contributions: Conceptualization, A.J.R. and L.N.M.; methodology, A.J.R. and L.N.M.; software, A.J.R.; validation, A.J.R.; formal analysis, A.J.R.; investigation, A.J.R.; data curation, A.J.R.; writing–original draft preparation, A.J.R.; writing–review and editing, L.N.M. and B.S.; visualization, A.J.R.; supervision, L.N.M. and B.S.; project administration, A.J.R. All authors have read and agreed to the published version of the manuscript.

Funding: This research did not receive specific funding from public, commercial, or not-for-profit agencies.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: All code and generated data are deposited at `research-data/presto/` (to be published on Zenodo with a DOI upon acceptance), including: (1) the Gaussian copula-based synthetic data generator with YAML configuration files; (2) the three regenerated synthetic domain datasets (170, 200, 150 releases); (3) feature engineering, ML training, PA-RFE, and autoregressive-feature analysis pipelines; (4) the bootstrap-confidence-interval and target-correlation ablation analysis; (5) the nested-CV hyperparameter tuning and Siegmund-style baseline reimplementation; (6) the System Availability ablation; (7) real-world validation adapters for TravisTorrent (including its permutation-importance companion script and the test-duration-exclusion robustness check), Mozilla Perfherder, GHALogs, and SQuaD, plus the bootstrap-CI extension, SQuaD CVE-count / Perfherder bug-enrichment scripts, and the SQuaD static-analysis (SonarQube/PMD/CK) enrichment pipeline added in this revision; and (8) all statistical validation and figure-generation scripts. **Software:** Python 3.14.5, scikit-learn 1.9.0, numpy 2.5.2, pandas 3.0.5, scipy 1.18.0 (pinned versions recorded in `ENVIRONMENT.txt`, matching Section 3.2.4). Random seed 42 fixed throughout for reproducibility. The synthetic generator is deterministic and designed as a reusable benchmark, allowing researchers to modify distribution parameters and domain profiles via YAML configuration.

Acknowledgments: During the preparation of this manuscript, the authors used Claude Code (Anthropic, Claude Sonnet 4.5/5 models) as an AI-assisted research and writing tool, for the following purposes: (1) drafting and revising manuscript prose across all sections; (2) writing the Python analysis scripts referenced throughout this paper (feature engineering, model training/evaluation, bootstrap confidence intervals, the target-correlation and System Availability ablations, nested cross-validation hyperparameter tuning, permutation importance, and the real-world dataset adapters for TravisTorrent, Mozilla Perfherder, GHALogs, and SQuaD), including reimplementing the PA-RFE algorithm and the TravisTorrent adapter from the authors' own pseudocode and methodology descriptions after the originals were found missing from the reproducibility package; (3) generating the figure-plotting scripts used to produce the paper's charts. All quantitative results reported in this paper were produced by executing this code against the underlying data (synthetic and real) with standard scientific Python libraries (scikit-learn, pandas, NumPy, SciPy); no numerical result, table entry, or figure value was generated or fabricated directly by the AI without underlying code execution. All AI-assisted code and prose were reviewed, verified, and edited by the authors, who take full responsibility for the content, correctness, and conclusions of this publication, including its methodological limitations and the honest reporting of findings (e.g., Section 4.1.6 and the "Near-Tautological Features" ablation in Section 5.1) that complicate rather than support the paper's own headline claims.

Conflicts of Interest: The authors declare no competing interests. Two authors are employed by organizations named in the affiliations (JPMorgan Chase & Co.; Lowe's Companies Inc.); no proprietary data, internal metrics, or confidential process/correlation analysis from either employer was used to inform this paper's 164-metric registry, the copula generator's 84 hand-specified correlation coefficients (including the 21 target correlations examined in the ablation study), or any other aspect of the synthetic data design. All metric definitions and correlation assumptions are grounded in publicly available DORA benchmarks and general SDLC domain knowledge, not employer-internal data.

Glossary

This glossary defines methodological terms and the four real-world datasets as used throughout this paper, distinct from the acronym-expansion list in the Abbreviations section that follows.

Methodology and evaluation terms

- Autoregressive (AR) features.** Rolling-window and lag statistics computed from the target variable's own prior values (e.g., 3-release rolling mean of System Uptime). Legitimate predictors after the shift fix (Section 3.1.3) that only uses strictly-prior observations, but they encode history-based persistence rather than SDLC process signal – this paper reports results with and without them throughout to keep the two apart.

- **Holdout R^2 .** Coefficient of determination computed once on a test partition held out from all training and model-selection, never re-used. This paper's pre-declared model-selection criterion (Section 3.2.4) in preference to mean CV R^2 . 1778
- **Cross-validation (CV) R^2 .** Mean R^2 across the folds of a `TimeSeriesSplit` applied to the training partition only, used for tuning and reported alongside holdout R^2 for transparency, not for model selection. 1779
- **Same-row (target) leakage.** A bug class where a feature computed for a given row inadvertently includes information from that same row's target value (e.g., a rolling window that has not been shifted before aggregation). This paper documents and fixes one such bug (`R2_LAG_SHIFT_FIX_EVIDENCE.md`) and treats its correction as load-bearing for every reported number. 1780
- **Gaussian copula.** The statistical technique used to generate PRESTO's synthetic SDLC metrics: it lets each of the 164 metrics keep its own realistic marginal distribution while reproducing a hand-specified correlation structure between metrics (Section 3.1.4). 1781
- **Target-correlation ablation.** An experiment that zeroes or randomly permutes the copula's hand-specified correlations between features and the target variable to test whether a model's predictive signal reflects genuine structure or recovery of what the generator was told to encode (Section 4.1.5). 1782
- **Permutation importance.** A model-agnostic feature-importance method that measures the drop in a fitted model's performance when a single feature's values are randomly shuffled, isolating that feature's marginal contribution. 1783
- **Bootstrap confidence interval.** A resampling-based uncertainty interval placed around a point estimate (here, holdout R^2) by repeatedly resampling the evaluation data with replacement and recomputing the statistic. 1784
- **Cross-sectional validation.** An evaluation design with one row per entity (e.g., one row per repository or project) and no within-entity time series, so there is no historical target value available to leak into any feature by construction (GHALogs, SQuAD). 1785
- **Within-project / within-signature temporal validation.** An evaluation design with a chronologically ordered split within a single project or signature's own history (TravisTorrent, Mozilla Perfherder), where AR-feature leakage is a live risk requiring the shift fix above. 1786
- **Selection (winner's-curse) bias.** The optimism introduced by choosing the best of several candidate models using the same metric later reported as the headline result – smaller than the optimism from full hyperparameter tuning, but not zero, and not corrected by the bootstrap CIs reported alongside it (those quantify sampling variance of the already-selected model, not the selection step itself). 1787
- **DORA metrics.** The key software-delivery performance metrics (deployment frequency, lead time for changes, change failure rate, time to restore service) from Google's DevOps Research and Assessment program, used to calibrate realistic value ranges in the synthetic data generator. 1788
- **CVE / CWE / CVSS.** Common Vulnerabilities and Exposures (an identifier for a specific disclosed security vulnerability), Common Weakness Enumeration (a taxonomy of the underlying weakness classes vulnerabilities fall into), and the Common Vulnerability Scoring System (the standard numeric severity score for a CVE) – used in the SQuAD CVE-count prediction analysis (Section 4.2.4.1). 1789

Real-world validation datasets 1820

- **TravisTorrent.** A dataset of Travis CI build logs joined with GitHub project metadata, covering 2.64 million builds across 1,359 open-source projects; used here for within-project build-duration prediction (Section 4.2.1). 1821
- **Mozilla Perfherder.** Mozilla's performance-regression tracking system (part of Treeherder), recording genuine per-push runtime performance measurements (page-load time, perceptual/contentful speed index) cross-referenced with human-confirmed regression alerts; used here for within-signature temporal validation. 1822
- **GHALogs.** A dataset of GitHub Actions workflow-run logs with per-step timing across 116k workflows and 25k repositories; used here for cross-sectional CI-duration prediction with zero historical-target leakage risk. 1823
- **SQuAD (Software Quality Dataset).** A multi-dimensional collection of process and static-analysis-tool metrics extracted from 450 mature open-source projects (Apache, Mozilla, FFmpeg, the Linux kernel, and others), submitted to the MSR Data and Tool Showcase Track; used here for cross-sectional defect-fix-rate prediction and, in the extended analysis, CVE-count prediction. The specific `process_metrics.csv` slice this paper downloads and uses covers 440 of the full dataset's 450 projects. 1824

Abbreviations 1837

The following abbreviations are used in this manuscript:

ACF	Autocorrelation Function
APM	Application Performance Monitoring
AR(1)	First-Order Autoregressive (process)
CDF	Cumulative Distribution Function
CI/CD	Continuous Integration / Continuous Delivery
CRM	Customer Relationship Management
CV	Cross-Validation
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DORA	DevOps Research and Assessment
GB	Gradient Boosting
KS	Kolmogorov-Smirnov (test)
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MLOps	Machine Learning Operations
OLS	Ordinary Least Squares
PA-RFE	Phase-Aware Recursive Feature Elimination
PCI-DSS	Payment Card Industry Data Security Standard
PCM	Palladio Component Model
PRESTO	Performance REgression from Software development Telemetry and Operations
PSI	Population Stability Index
RF	Random Forest
RFE	Recursive Feature Elimination
RMSE	Root Mean Squared Error
SaaS	Software as a Service
SDLC	Software Development Lifecycle
SHAP	SHapley Additive exPlanations
SLO	Service Level Objective
SLOC	Source Lines of Code
SPE	Software Performance Engineering
SQuAD	Software Quality Dataset
SRE	Site Reliability Engineering
UAT	User Acceptance Testing
UML	Unified Modeling Language

References

1. Lazowska, E.D.; Zahorjan, J.; Graham, G.S.; Sevcik, K.C. *Quantitative System Performance: Computer System Analysis Using Queueing Network Models*; Prentice-Hall, 1984.
2. Reiser, M.; Lavenberg, S.S. Mean-value analysis of closed multichain queueing networks. *Journal of the ACM* **1980**, *27*, 313–322. <https://doi.org/10.1145/322186.322195>.
3. Ferrari, D. *Computer Systems Performance Evaluation*; Prentice-Hall, 1978.
4. Sauer, C.H.; Chandy, K.M. *Computer Systems Performance Modeling*; Prentice-Hall, 1981.
5. Jain, R. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*; Wiley, 1991.
6. Smith, C.U. *Performance Engineering of Software Systems*; Addison-Wesley, 1990.
7. Smith, C.U.; Williams, L.G. *Performance Solutions: A Practical Guide to Creating Responsive, Scalable Software*; Addison-Wesley, 2002.
8. Balsamo, S.; Di Marco, A.; Inverardi, P.; Simeoni, M. Model-based performance prediction in software development: A survey. *IEEE Transactions on Software Engineering* **2004**, *30*, 295–310. <https://doi.org/10.1109/TSE.2004.9>.
9. Cortellessa, V.; Di Marco, A.; Inverardi, P. *Model-Based Software Performance Analysis*; Springer, 2011. <https://doi.org/10.1007/978-3-642-13621-4>.

10. Jiang, Z.M.; Hassan, A.E. A Survey on Load Testing of Large-Scale Software Systems. *IEEE Transactions on Software Engineering* **2015**, *41*, 1091–1118. <https://doi.org/10.1109/TSE.2015.2445340>. 1857
11. Nguyen, T.H.D.; Adams, B.; Jiang, Z.M.; Hassan, A.E.; Nasser, M.; Flora, P. Automated Detection of Performance Regressions Using Statistical Process Control Techniques. In Proceedings of the Proceedings of the 3rd ACM/SPEC International Conference on Performance Engineering (ICPE), 2012, pp. 299–310. <https://doi.org/10.1145/2188286.2188344>. 1858
12. Foo, K.C.; Jiang, Z.M.; Adams, B.; Hassan, A.E.; Zou, Y.; Flora, P. Mining Performance Regression Testing Repositories for Automated Performance Analysis. In Proceedings of the Proceedings of the 10th International Conference on Quality Software (QSIC), 2010, pp. 32–41. <https://doi.org/10.1109/QSIC.2010.35>. 1859
13. HP Labs. Predicting Performance in Distributed Enterprise Applications. Technical Report HPL-2006-76, HP Labs, 2006. 1860
14. Weyuker, E.J.; Vokolos, F.I. Experience with Performance Testing of Software Systems: Issues, an Approach, and Case Study. *IEEE Transactions on Software Engineering* **2000**, *26*, 1147–1156. <https://doi.org/10.1109/32.888628>. 1861
15. Liu, Y.; Fekete, A.; Gorton, I. Design-level performance prediction of component-based applications. *IEEE Transactions on Software Engineering* **2005**, *31*, 928–941. <https://doi.org/10.1109/TSE.2005.127>. 1862
16. Pooley, R.J.; King, P. The Unified Modelling Language and performance engineering. *IEE Proceedings — Software* **1999**, *146*, 2–10. <https://doi.org/10.1049/ip-sen:19990151>. 1863
17. Menascé, D.A.; Almeida, V.A.F.; Dowdy, L.W. *Capacity Planning for Web Services: Metrics, Models, and Methods*; Prentice Hall, 2004. 1864
18. Kant, K. *Introduction to Computer System Performance Evaluation*; McGraw-Hill, 1992. 1865
19. Woodside, C.M.; Franks, G.; Petriu, D.C. The future of software performance engineering. In Proceedings of the Proceedings of the Future of Software Engineering (FOSE), 2007, pp. 171–187. <https://doi.org/10.1109/FOSE.2007.32>. 1866
20. Koziolok, H. Performance evaluation of component-based software systems: A survey. *Performance Evaluation* **2010**, *67*, 634–658. <https://doi.org/10.1016/j.peva.2009.07.007>. 1867
21. Becker, S.; Koziolok, H.; Reussner, R. The Palladio Component Model for Model-Driven Performance Prediction. *Journal of Systems and Software* **2009**, *82*, 3–22. <https://doi.org/10.1016/j.jss.2008.03.066>. 1868
22. Jin, D.; Qu, X.; Cohen, M.B.; Robinson, B. Configurations everywhere: Implications for testing and debugging in practice. In Proceedings of the Proceedings of the International Conference on Software Engineering (ICSE), 2014. <https://doi.org/10.1145/2591062.2591191>. 1869
23. Chen, L.; Babar, M.A.; Cawley, C. A status report on the evaluation of variability management approaches. In Proceedings of the Proceedings of the International Conference on Evaluation and Assessment in Software Engineering, 2009. <https://doi.org/10.14236/EWIC/EASE2009.14>. 1870
24. Chidamber, S.R.; Kemerer, C.F. A metrics suite for object oriented design. *IEEE Transactions on Software Engineering* **1994**, *20*, 476–493. <https://doi.org/10.1109/32.295895>. 1871
25. McCabe, T.J. A complexity measure. *IEEE Transactions on Software Engineering* **1976**, *SE-2*, 308–320. <https://doi.org/10.1109/TSE.1976.233837>. 1872
26. Boehm, B.W.; Abts, C.; Brown, A.W.; Chulani, S.; Clark, B.K.; Horowitz, E.; Madachy, R.; Reifer, D.J.; Steece, B. *Software Cost Estimation with COCOMO II*; Prentice Hall, 2000. 1873
27. Ostrand, T.J.; Weyuker, E.J. The distribution of faults in a large industrial software system. In Proceedings of the Proceedings of the International Symposium on Software Testing and Analysis (ISSTA), 2002, pp. 55–64. <https://doi.org/10.1145/566172.566181>. 1874
28. Nagappan, N.; Ball, T.; Zeller, A. Mining metrics to predict component failures. In Proceedings of the Proceedings of the International Conference on Software Engineering (ICSE), 2006, pp. 452–461. <https://doi.org/10.1145/1134285.1134349>. 1875
29. Lessmann, S.; Baesens, B.; Mues, C.; Pietsch, S. Benchmarking Classification Models for Software Defect Prediction: A Proposed Framework and Novel Findings. *IEEE Transactions on Software Engineering* **2008**, *34*, 485–496. <https://doi.org/10.1109/TSE.2008.35>. 1876
30. Kamei, Y.; Shihab, E.; Adams, B.; Hassan, A.E.; Mockus, A.; Sinha, A.; Ubayashi, N. A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering* **2013**, *39*, 757–773. <https://doi.org/10.1109/TSE.2012.70>. 1877
31. Arcoverde, R.; Garcia, A.; Figueiredo, E. Understanding the longevity of code smells: Preliminary results of an explanatory survey. In Proceedings of the Proceedings of the International Workshop on Refactoring Tools, 2011, pp. 33–36. <https://doi.org/10.1145/1984732.1984740>. 1878
32. Li, Z.; Avgeriou, P.; Liang, P. A systematic mapping study on technical debt and its management. *Journal of Systems and Software* **2015**, *101*, 193–220. <https://doi.org/10.1016/j.jss.2014.12.027>. 1879
33. Forsgren, N.; Humble, J.; Kim, G. *Accelerate: The Science of Lean Software and DevOps*; IT Revolution Press, 2018. 1880
34. Google Cloud. Accelerate State of DevOps Report 2024. Technical report, DORA, 2024. 1881
35. Hilton, M.; Tunnell, T.; Huang, K.; Marinov, D.; Dig, D. Usage, costs, and benefits of continuous integration in open-source projects. In Proceedings of the Proceedings of the International Conference on Automated Software Engineering (ASE), 2016, pp. 426–437. <https://doi.org/10.1145/2970276.2970358>. 1882

36. Vasilescu, B.; Yu, Y.; Wang, H.; Devanbu, P.; Filkov, V. Quality and productivity outcomes relating to continuous integration in GitHub. In Proceedings of the Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE), 2015, pp. 805–816. <https://doi.org/10.1145/2786805.2786850>. 1911–1913
37. Gallaba, K.; McIntosh, S. Use and misuse of continuous integration features: An empirical study of projects that (mis)use Travis CI. *IEEE Transactions on Software Engineering* **2020**, *46*, 33–50. <https://doi.org/10.1109/TSE.2018.2838131>. 1914–1915
38. Ghaleb, T.A.; da Costa, D.A.; Zou, Y. An Empirical Study of the Long Duration of Continuous Integration Builds. *Empirical Software Engineering* **2019**, *24*, 2102–2139. <https://doi.org/10.1007/s10664-019-09695-9>. 1916–1917
39. Beller, M.; Gousios, G.; Zaidman, A. Oops, My Tests Broke the Build: An Explorative Analysis of Travis CI with GitHub. In Proceedings of the Proceedings of the 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 356–367. <https://doi.org/10.1109/MSR.2017.62>. 1918–1920
40. Saidani, I.; Ouni, A.; Mkaouer, M.W. Improving the Prediction of Continuous Integration Build Failures Using Deep Learning. *Automated Software Engineering* **2022**, *29*, 1–61. <https://doi.org/10.1007/s10515-021-00319-5>. 1921–1922
41. Riungu-Kalliosaari, L.; Mäkinen, S.; Lwakatare, L.E.; Tiihonen, J.; Männistö, T. DevOps adoption benefits and challenges in practice: A case study. In Proceedings of the Proceedings of the International Conference on Product-Focused Software Process Improvement (PROFES), 2016. https://doi.org/10.1007/978-3-319-49094-6_44. 1923–1925
42. Amaro, R.M.D.; Pereira, R.; Mira da Silva, M. DevOps Metrics and KPIs: A Multivocal Literature Review. *ACM Computing Surveys* **2024**, *56*. <https://doi.org/10.1145/3652508>. 1926–1927
43. Brunnert, A.; van Hoorn, A.; Willnecker, F.; Danciu, A.; Hasselbring, W.; Heger, C.; Herbst, N.; Kounev, S.; Koziol, H.; Kroß, J.; et al. Performance-oriented DevOps: A Research Agenda. In Proceedings of the Companion Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE), 2015. 1928–1930
44. Fallahzadeh, E.; Bavand, A.H.; Rigby, P.C. Accelerating Continuous Integration with Parallel Batch Testing. In Proceedings of the Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2023. <https://doi.org/10.1145/3611643.3616255>. 1931–1933
45. Menzies, T.; Greenwald, J.; Frank, A. Data mining static code attributes to learn defect predictors. *IEEE Transactions on Software Engineering* **2007**, *33*, 2–13. <https://doi.org/10.1109/TSE.2007.256941>. 1934–1935
46. Hall, T.; Beecham, S.; Bowes, D.; Gray, D.; Counsell, S. A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering* **2012**, *38*, 1276–1304. <https://doi.org/10.1109/TSE.2011.103>. 1936–1937
47. Zhang, J.; Wang, X.; Zhang, H.; Sun, H.; Wang, K.; Liu, X. A Novel Neural Source Code Representation Based on Abstract Syntax Tree. In Proceedings of the Proceedings of the 41st International Conference on Software Engineering (ICSE), 2019, pp. 783–794. <https://doi.org/10.1109/ICSE.2019.00086>. 1938–1940
48. Allamanis, M.; Barr, E.T.; Devanbu, P.; Sutton, C. A survey of machine learning for big code and naturalness. *ACM Computing Surveys* **2018**, *51*, 1–37. <https://doi.org/10.1145/3212695>. 1941–1942
49. Chen, B.; Chen, L.; Zhang, C.; Peng, X. BuildFast: History-Aware Build Outcome Prediction for Fast Feedback and Reduced Cost in Continuous Integration. In Proceedings of the Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2020, pp. 42–53. <https://doi.org/10.1145/3324884.3416616>. 1943–1945
50. Mishra, L.N.; Rangari, A.; Nagrare, S.; Nayak, S.K. A Taxonomy for Detecting and Preventing Temporal Data Leakage in Machine Learning-Based Build Prediction: A Dual-Platform Empirical Validation. *PLOS ONE* **2026**, *21*, e0340167. <https://doi.org/10.1371/journal.pone.0340167>. 1946–1948
51. Wu, Y.; Chai, B.; Li, Y.; Liu, B.; Li, J.; Yang, Y.; Jiang, W. An Empirical Study on Change-Induced Incidents of Online Service Systems. In Proceedings of the Proceedings of the IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), 2023, pp. 234–245. <https://doi.org/10.1109/ICSE-SEIP58684.2023.00027>. 1949–1951
52. Nistor, A.; Song, L.; Marinov, D.; Lu, S. Toddler: Detecting performance problems via similar memory-access patterns. In Proceedings of the Proceedings of the International Conference on Software Engineering (ICSE), 2013, pp. 562–571. <https://doi.org/10.1109/ICSE.2013.6606602>. 1952–1954
53. Siegmund, N.; Grebhahn, A.; Apel, S.; Kästner, C. Performance-Influence Models for Highly Configurable Systems. In Proceedings of the Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE), 2015, pp. 284–294. <https://doi.org/10.1145/2786805.2786845>. 1955–1957
54. Ha, H.; Zhang, H. DeepPerf: Performance Prediction for Configurable Software with Deep Sparse Neural Network. In Proceedings of the Proceedings of the 41st International Conference on Software Engineering (ICSE), 2019, pp. 1095–1106. <https://doi.org/10.1109/ICSE.2019.00113>. 1958–1960
55. Nair, V.; Yu, Z.; Menzies, T.; Siegmund, N.; Apel, S. Finding Faster Configurations Using FLASH. *IEEE Transactions on Software Engineering* **2020**, *46*, 794–811. <https://doi.org/10.1109/TSE.2018.2870895>. 1961–1962
56. Guo, J.; Yang, D.; Siegmund, N.; Apel, S.; Sarkar, A.; Valov, P.; Czarnecki, K.; Wasowski, A.; Yu, H. Data-efficient Performance Learning for Configurable Systems. *Empirical Software Engineering* **2018**, *23*, 1826–1867. <https://doi.org/10.1007/s10664-017-9573-6>. 1963–1965

57. Jamshidi, P.; Velez, M.; Kästner, C.; Siegmund, N.; Kawthekar, P. Transfer Learning for Improving Model Predictions in Highly Configurable Software. In Proceedings of the Proceedings of the 12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2017. <https://doi.org/10.1109/SEAMS.2017.11>.
58. Jamshidi, P.; Velez, M.; Kästner, C.; Siegmund, N. Learning to sample: Exploiting similarities across environments to learn performance models for configurable systems. In Proceedings of the Proceedings of the Joint Meeting on Foundations of Software Engineering (ESEC/FSE), 2018, pp. 71–82. <https://doi.org/10.1145/3236024.3236074>.
59. Maros, A.; Murai, F.; da Silva, A.P.C.; Almeida, J.M.; Lattuada, M.; Gianniti, E.; Hosseini, M.; Ardagna, D. Machine Learning for Performance Prediction of Spark Cloud Applications. In Proceedings of the Proceedings of the IEEE 12th International Conference on Cloud Computing (CLOUD), 2019, pp. 99–106. <https://doi.org/10.1109/CLOUD.2019.00028>.
60. Tantithamthavorn, C.; McIntosh, S.; Hassan, A.E.; Matsumoto, K. An Empirical Comparison of Model Validation Techniques for Defect Prediction Models. *IEEE Transactions on Software Engineering* **2017**, *43*, 1–18. <https://doi.org/10.1109/TSE.2016.2584050>.
61. Liao, L.; Eismann, S.; Li, H.; Bezemer, C.P.; Costa, D.E.; van Hoorn, A.; Shang, W. Early Detection of Performance Regressions by Bridging Local Performance Data and Architectural Models. In Proceedings of the Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE), 2025. <https://doi.org/10.1109/ICSE55347.2025.00026>.
62. Biringa, C.; Kul, G. PACE: A Program Analysis Framework for Continuous Performance Prediction. *ACM Transactions on Software Engineering and Methodology* **2024**. <https://doi.org/10.1145/3637230>.
63. Mboweni, T.; Masombuka, T.; Dongmo, C. A Systematic Review of Machine Learning DevOps. In Proceedings of the Proceedings of the International Conference on Electrical, Computer and Energy Technologies (ICECET), 2022. <https://doi.org/10.1109/ICECET55527.2022.9872968>.
64. Mohapatra, H.; Pramanik, S.; Mishra, S.R. Revolutionizing Software Development: The Transformative Influence of Machine Learning Integrated SDLC Model. In *Boosting Software Development Using Machine Learning*; Springer, 2025; Vol. 7, *Artificial Intelligence-Enhanced Software and Systems Engineering*. https://doi.org/10.1007/978-3-031-88188-6_3.
65. Behera, A.K.; Chaudhury, P.; Dash, C.S.K. A Comprehensive Survey on Intelligent Software Reliability Prediction. *Discover Computing* **2025**. <https://doi.org/10.1007/s10791-025-09597-z>.
66. Zhou, X.; Peng, X.; Xie, T.; Sun, J.; Ji, C.; Liu, D.; Xiang, Q.; He, C. Latent Error Prediction and Fault Localization for Microservice Applications by Learning from System Trace Logs. In Proceedings of the Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2019, pp. 683–694. <https://doi.org/10.1145/3338906.3338961>.
67. Dang, Y.; Lin, Q.; Huang, P. AIOps: Real-World Challenges and Research Innovations. In Proceedings of the Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Companion (ICSE-Companion), 2019, pp. 4–5. <https://doi.org/10.1109/ICSE-Companion.2019.00023>.
68. Notaro, P.; Cardoso, J.; Gerndt, M. A Survey of AIOps Methods for Failure Management. *ACM Transactions on Intelligent Systems and Technology* **2021**, *12*, 1–45. <https://doi.org/10.1145/3483424>.
69. Diaz-De-Arcaya, J.; Torre-Bastida, A.I.; Zárate, G.; Miñón, R.; Almeida, A. A Joint Study of the Challenges, Opportunities, and Roadmap of MLOps and AIOps: A Systematic Survey. *ACM Computing Surveys* **2024**, *56*, 1–30. <https://doi.org/10.1145/3625289>.
70. Basiri, A.; Behnam, N.; de Rooij, R.; Hochstein, L.; Kosewski, L.; Reynolds, J.; Rosenthal, C. Chaos Engineering. *IEEE Software* **2016**, *33*, 35–41. <https://doi.org/10.1109/MS.2016.60>.
71. Rosenthal, C.; Jones, N. *Chaos Engineering: System Resiliency in Practice*; O'Reilly Media, 2020.
72. Beller, M.; Gousios, G.; Zaidman, A. TravisTorrent: Synthesizing Travis CI and GitHub for Full-Stack Research on Continuous Integration. In Proceedings of the Proceedings of the 14th International Conference on Mining Software Repositories (MSR), 2017, pp. 447–450. <https://doi.org/10.1109/MSR.2017.24>.
73. Nelsen, R.B. *An Introduction to Copulas*, 2nd ed.; Springer, 2006. <https://doi.org/10.1007/0-387-28678-0>.
74. Higham, N.J. Computing the Nearest Correlation Matrix—A Problem from Finance. *IMA Journal of Numerical Analysis* **2002**, *22*, 329–343. <https://doi.org/10.1093/imanum/22.3.329>.
75. Guyon, I.; Weston, J.; Barnhill, S.; Vapnik, V. Gene Selection for Cancer Classification using Support Vector Machines. *Machine Learning* **2002**, *46*, 389–422. <https://doi.org/10.1023/A:1012487302797>.
76. Breiman, L. Random Forests. *Machine Learning* **2001**, *45*, 5–32. <https://doi.org/10.1023/A:1010933404324>.
77. Friedman, J.H. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics* **2001**, *29*, 1189–1232. <https://doi.org/10.1214/aos/1013203451>.
78. Gama, J.a.; Žliobaitė, I.; Bifet, A.; Pechenizkiy, M.; Bouchachia, A. A Survey on Concept Drift Adaptation. *ACM Computing Surveys* **2014**, *46*, 1–37. <https://doi.org/10.1145/2523813>.
79. Besbes, M.B.; Damasceno Costa, D.E.; Mujahid, S.; Castelluccio, M.; Mierzwinski, G. Mozilla Performance Time Series Dataset, 2025. CC-BY-4.0, <https://doi.org/10.5281/zenodo.14642238>.
80. Moriconi, F.; Durieux, T.; Falleri, J.R.; Troncy, R.; Francillon, A. GHALogs: Large-Scale Dataset of GitHub Actions Runs, 2024. CC-BY-SA-4.0, <https://doi.org/10.5281/zenodo.10154920>.

81. Robredo, M.; Esposito, M.; Taibi, D.; Peñaloza, R.; Lenarduzzi, V. SQuaD: The Software Quality Dataset, 2025. CC-BY-4.0. Submitted to the MSR Data and Tool Showcase Track., <https://doi.org/10.5281/zenodo.17566690>. 2021
82. Shankar, S.; Garcia, R.; Hellerstein, J.M.; Parameswaran, A.G. Operationalizing Machine Learning: An Interview Study. *Journal of Machine Learning Research* **2024**, *25*, 1–45. 2023

Short Biography of Authors



Amit J Rangari is a Lead Software Engineer in DevOps and Performance at JPMorgan Chase & Co., Atlanta, GA, USA. A published researcher in software engineering, his work focuses on CI/CD automation, platform resilience, containerization, and cloud-native architectures. With over 20 years of experience across the banking, telecommunications, and pharmaceutical sectors, he has led large-scale DevOps transformations for Fortune 500 clients. He is a certified AWS Cloud Practitioner and serves as a peer reviewer for IEEE-affiliated journals in artificial intelligence, healthcare systems, and precision agriculture. He combines his technical expertise with a commitment to advancing industry practices, regularly contributing to academic discourse through publications and presentations.



Lalit N. Mishra is a Sr. Manager of Software Engineering at Lowe's Companies Inc., Mooresville, NC, USA, where he leads three cross-functional teams building AI-driven retail platforms. He received the M.S. degree in Computer Science from Manipal University, India, in 2013. He is the author of *Spring, Hibernate, Data Modeling, REST and TDD* (Amazon, 2015) and has published in IEEE Access, including the Retail Resilience Engine framework (2025) and the FraudFusion ensemble fraud-detection system (2026). An IEEE Senior Member, he serves as a peer reviewer for PLOS ONE and IGI Global journals. He has also served as a judge and mentor at multiple university hackathons. His research interests include explainable artificial intelligence, retail demand forecasting, ensemble machine learning, and agentic AI systems.



Biswaranjan Senapati (Senior Member, IEEE) is with the Department of Computer Science, University of Arkansas at Little Rock, Little Rock, AR, USA. He is a Platinum-Level ERP-SAP Consultant with more than 22 years of industry experience in domain consulting, business consulting, and solution architecture for large-scale ERP system design, development, implementation, rollout, support, and business process re-engineering capacity in the supply chain, chemicals, hi-tech, CPG, pharmaceuticals, wholesale, fashion retail, health care, and manufacturing domains. He has previously worked at Parker Hannifin, Home Depot, SONY, Gordon Foods, Constellation Brands, Olin Corporations, HPE, Ecolabs, TSC, HanesBrands, PVH, Applexus, ECCO Denmark, Mindtree Germany, Ciber USA, Cognizant, Capgemini, and Deloitte Consulting, and the SAP Group of Consulting Companies.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.